MLOps CI/CD Pipeline with GitHub Actions, Pulumi, and Grafana

MLOps CI/CD Pipeline with GitHub Actions, Pulumi, and Grafana

Introduction

This comprehensive MLOps pipeline demonstrates a modern cloud-native approach to machine learning operations using Infrastructure as Code (IaC) and containerized microservices architecture. The project implements a complete CI/CD pipeline that automatically builds, tests, and deploys ML services to AWS with real-time monitoring capabilities.

Our Chosen CI/CD Approach

Infrastructure-First, Clean Slate Deployment Strategy

This project adopts a "destroy and recreate" approach rather than traditional incremental updates. Here's why this approach was chosen:

🔄 Complete Infrastructure Refresh: On every deployment, Pulumi completely destroys and recreates the entire AWS infrastructure, ensuring a clean, consistent environment free from configuration drift.

🐳 Container-First Architecture: Both microservices are fully containerized using Docker, ensuring consistency across development, testing, and production environments.

🚀 Parallel Processing Pipeline: The CI/CD workflow runs tests in parallel for both services, then sequences infrastructure provisioning, image building, and deployment for optimal efficiency.

🔍 Built-in Observability: Prometheus and Grafana are automatically deployed alongside the services, providing immediate visibility into system health and performance metrics.

Why This Approach Works

Simplicity Over Complexity: While blue-green or rolling deployments might seem more sophisticated, this approach prioritizes reliability and predictability over zero-downtime

deployments. For ML services where model consistency is crucial, a brief deployment window is often acceptable in exchange for guaranteed clean state.

Perfect for ML Workloads: Machine learning services benefit from this approach because:

- Model versions are clearly separated
- No risk of mixed configurations between deployments
- Easy rollback by simply redeploying previous version
- Consistent environment for reproducible predictions

Architecture Overview

The pipeline follows a microservices pattern with two specialized services:

- ML Inference Service (Port 8001): Handles prediction requests with built-in metrics
- Data Ingestion Service (Port 8002): Processes incoming data with validation and storage
- Monitoring Stack: Prometheus for metrics collection and Grafana for visualization
- Infrastructure Management: Pulumi provides declarative AWS resource management

Key Benefits of This Implementation

Reproducible Deployments: Every deployment creates identical infrastructure

Comprehensive Testing: Automated testing ensures code quality before deployment

Real-time Monitoring: Immediate visibility into service health and performance

Scalable Architecture: Easy to add new services or modify existing ones

Cost-Effective: Uses AWS free tier resources (t2.micro instances)

Developer-Friendly: Simple git push triggers entire deployment workflow

This approach is particularly well-suited for teams prioritizing reliability and simplicity over complex deployment strategies, making it ideal for ML operations where consistency and observability are paramount.

Prerequisites

Install these tools on your local machine:

1. Git: Version control system

```
# Check if installed
git --version
# Install on Ubuntu/Debian
sudo apt-get update
sudo apt-get install git
# Install on macOS
brew install git
```

2.
3. Docker: For containerization

```
# Check if installed
docker --version
```

4. # Install Docker Desktop from https://www.docker.com/products/docker-desktop/
5. Python 3.8+: Programming language

```
# Check if installed
python3 --version
# Install on Ubuntu/Debian
sudo apt-get install python3 python3-pip
```

6.
7. Node.js and npm: Required for Pulumi

```
# Check if installed
node --version
npm --version
# Install using NodeSource repository
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
```

8.
9. Pulumi: Infrastructure as Code tool

```
# Install Pulumi
curl -fsSL https://get.pulumi.com | sh
# Add to PATH (add this to your .bashrc or .zshrc)
export PATH=$PATH:$HOME/.pulumi/bin
# Verify installation
```

10. pulumi version
11. AWS CLI: Command-line interface for AWS

```
# Install AWS CLI
```

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
# Verify installation
aws --version
```

Project Structure

Create the following directory structure for your project:

```
mlops-pipeline/
├── services/
│   ├── ml-inference/
│   │   ├── app.py
│   │   ├── __init__.py
│   │   ├── requirements.txt
│   │   ├── Dockerfile
│   │   └── tests/
│   │       ├── test_app.py
│   │       ├── __init__.py
│   └── data-ingestion/
│   │   ├── app.py
│   │   ├── __init__.py
│   │   ├── requirements.txt
│   │   ├── Dockerfile
│   │   └── tests/
│   │       ├── test_app.py
│   │       ├── __init__.py
├── infrastructure/
│   ├── Pulumi.yaml
│   ├── Pulumi.dev.yaml
│   ├── __main__.py
│   └── requirements.txt
├── monitoring/
│   ├── prometheus/
│   │   ├── prometheus.yml
│   │   └── alerts.yml
│   └── grafana/
│   │   ├── dashboards/
│   │   │   └── mlops-dashboard.json
│   │   └── provisioning/
│   │       ├── datasources/
│   │       │   └── prometheus.yml
```

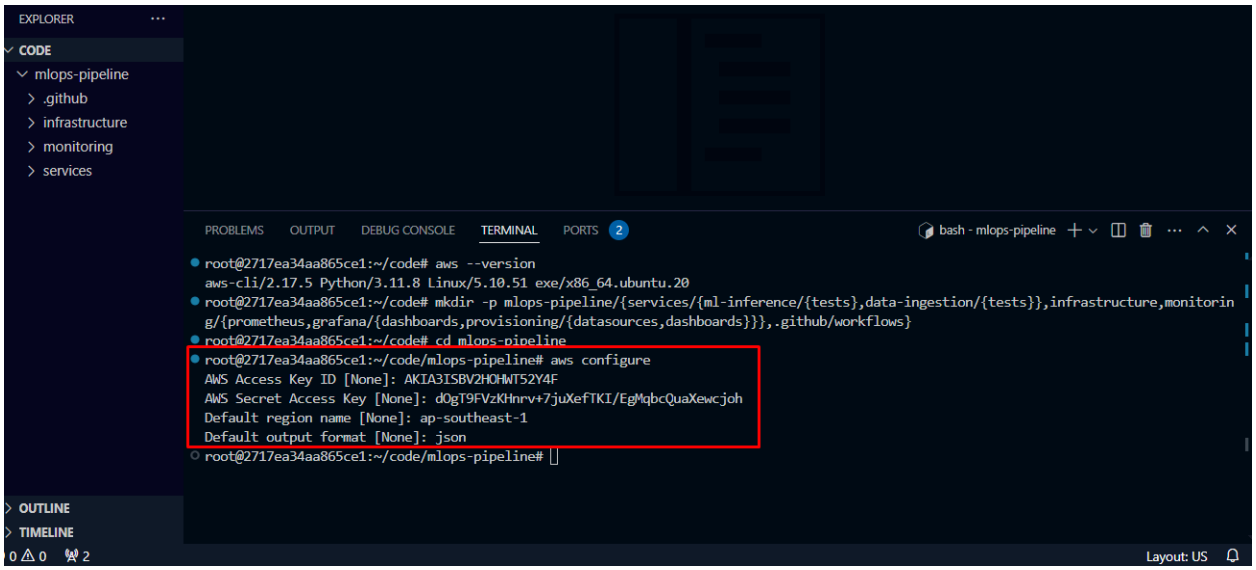
```
│           └── dashboards/
│                └── dashboard.yml
├── .github/
│   └── workflows/
│       └── deploy.yml
├── docker-compose.yml
├── README.md
└── .gitignore
```

Step 1: Setting Up AWS Sandbox

1.1 Configure AWS CLI

Using the provided sandbox credentials:

```
# Configure AWS CLI with your sandbox credentials
aws configure

# Enter the following when prompted:
AWS Access Key ID:  Your provided Access Key
AWS Secret Access Key:  Your provided Secret Key
Default region name: ap-southeast-1
Default output format: json

# Verify configuration
aws sts get-caller-identity
```



1.2 Create .gitignore

Create `.gitignore` file:

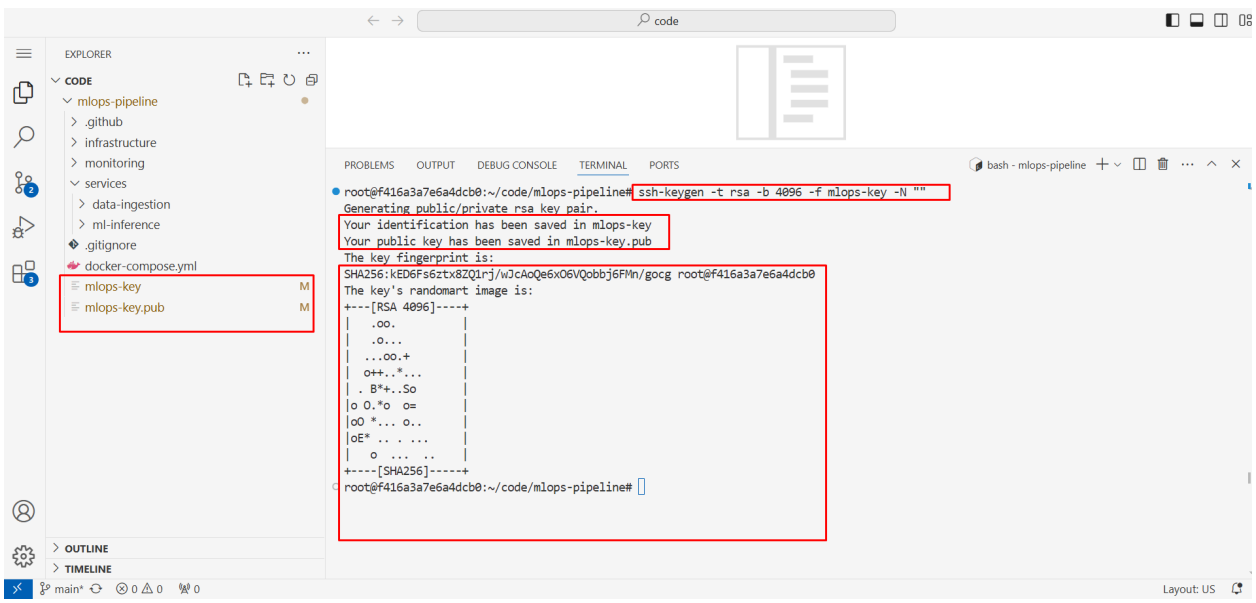
```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
env/
venv/
ENV/
.env

# Pulumi
*.pyc
Pulumi.*.yaml
!Pulumi.yaml

# AWS
.aws/

# IDE
.vscode/
.idea/
*.swp
*.swo
```

```
# OS
.DS_Store
Thumbs.db

# Logs
*.log

# Docker
.docker/
```

create public and private key for ec2.

```
ssh-keygen -t rsa -b 4096 -f mlops-key -N ""
```

It will create a public and private key name mlops-key



Step 2: Creating the Microservices

2.1 ML Inference Service

Create `services/ml-inference/app.py`

```
from flask import Flask, request, jsonify
import numpy as np
import logging
from prometheus_client import Counter, Histogram, generate_latest
CONTENT_TYPE_LATEST
```

```
import time

# Configure logging
logging basicConfig level=logging INFO
logger = logging getLogger __name__

app = Flask __name__

# Prometheus metrics
prediction_counter = Counter 'ml_predictions_total'  'Total number of
predictions made'
prediction_duration = Histogram 'ml_prediction_duration_seconds'
'Time spent processing prediction'

# Simple dummy model (replace with your actual model)
class DummyModel
    def predict self  features
        # Simulate some processing time
        time sleep 0.1
        # Return dummy prediction
        return np random random

model = DummyModel

           '/health'  methods= 'GET'
def health
    """Health check endpoint"""
    return jsonify  'status'  'healthy'  'service'  'ml-inference'
200

           '/predict'  methods= 'POST'
def predict
    """Prediction endpoint"""
    start_time = time time

    try
        # Get features from request
        data = request get_json
        if not data or 'features' not in data
            return jsonify  'error'  'No features provided'    400

        features = data 'features'
        logger info f"Received prediction request with features:
 features "
```

```
        # Make prediction
        prediction = model.predict(features)

        # Update metrics
        prediction_counter.inc()
        prediction_duration.observe(time.time() - start_time)

        response = {
            'prediction': float(prediction[0]),
            'model_version': '1.0.0',
            'timestamp': time.time()
        }

        logger.info(f"Prediction completed: {response}")
        return jsonify(response), 200

    except Exception as e:
        logger.error(f"Prediction error: {str(e)}")
        return jsonify({'error': str(e)}), 500

@app.route('/metrics', methods=['GET'])
def metrics():
    """Prometheus metrics endpoint"""
    try:
        # Return metrics with proper content type
        return generate_latest(), 200, {'Content-Type': CONTENT_TYPE_LATEST}
    except Exception as e:
        logger.error(f"Metrics error: {str(e)}")
        return "Metrics unavailable", 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8001, debug=False)
```

This is a Flask web API that serves machine learning predictions with built-in monitoring. It has three main endpoints: `/health` for checking if the service is running, `/predict` for making ML predictions (currently using a dummy model that returns random numbers), and `/metrics` for exposing performance data to Prometheus. When you send features to the `/predict` endpoint, it processes them through the model, tracks how long it takes, counts the number of predictions made, and returns a JSON response with the prediction result, model version, and

timestamp. The service includes proper error handling, logging, and automatic metrics collection so Prometheus can monitor its performance in real-time.

In simple terms- It's a web service that takes data, runs it through an ML model, and gives you back a prediction - while automatically tracking its own performance for monitoring.

Create `services/ml-inference/requirements.txt`:

```
flask==2.3.3
numpy==1.24.3
prometheus-client==0.17.1
gunicorn==21.2.0
```

Create `services/ml-inference/tests/test_app.py`:

```
import unittest
import json
import sys
import os

# Add the parent directory to Python path so we can import app
sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

try:
    from app import app
except ImportError:
    # If direct import fails, try relative import
    import app as app_module
    app = app_module.app

class TestMLInference(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True

    def test_health_endpoint(self):
        response = self.app.get('/health')
        self.assertEqual(response.status_code, 200)
        data = json.loads(response.data)
        self.assertEqual(data['status'], 'healthy')

    def test_predict_endpoint(self):
        test_data = {'features': [1.0, 2.0, 3.0]}
```

```
        response = self.app.post('/predict',
                                 data=json.dumps(test_data),
                                 content_type='application/json')
        self.assertEqual(response.status_code, 200)
        data = json.loads(response.data)
        self.assertIn('prediction', data)
        self.assertIn('model_version', data)

    def test_predict_no_features(self):
        response = self.app.post('/predict',
                                 data=json.dumps({}),
                                 content_type='application/json')
        self.assertEqual(response.status_code, 400)

if __name__ == '__main__':
    unittest.main()
```

This is an automated test suite that validates the ML inference service works correctly before deployment. It uses Python's unittest framework to create a test client that simulates HTTP requests to the Flask app without actually starting a web server. The tests cover three key scenarios: checking that the `/health` endpoint returns a 200 status with "healthy" response, verifying the `/predict` endpoint accepts valid feature data and returns a proper prediction with model version, and ensuring the service correctly rejects requests with missing features by returning a 400 error. The test setup handles import path issues to reliably load the Flask app from different directory structures, making it robust for CI/CD environments.

2.2 Data Ingestion Service

Create `services/data-ingestion/app.py`:

```
from flask import Flask, request, jsonify
import logging
import json
import time
from datetime import datetime
from prometheus_client import Counter, Histogram, generate_latest,
CONTENT_TYPE_LATEST
import os

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
```

```
app = Flask(__name__)

# Prometheus metrics
ingestion_counter = Counter('data_ingestion_total', 'Total number of
data ingested')
ingestion_duration = Histogram('data_ingestion_duration_seconds',
'Time spent ingesting data')
data_size_histogram = Histogram('data_size_bytes', 'Size of ingested
data in bytes')

# Data storage (in production, use a database or message queue)
data_store = []

@app.route('/health', methods=['GET'])
def health():
    """Health check endpoint"""
    return jsonify({'status': 'healthy', 'service':
'data-ingestion'}), 200

@app.route('/ingest', methods=['POST'])
def ingest():
    """Data ingestion endpoint"""
    start_time = time.time()

    try:
        # Get data from request
        data = request.get_json()
        if not data:
            return jsonify({'error': 'No data provided'}), 400

        # Add metadata
        ingestion_record = {
            'data': data,
            'timestamp': datetime.utcnow().isoformat(),
            'source_ip': request.remote_addr,
            'id': len(data_store) + 1,


        # Simulate data processing
        time.sleep(0.05)

        # Store data (in production, save to database)
        data_store.append(ingestion_record)
```

```
        # Update metrics
        ingestion_counter.inc()
        ingestion_duration.observe(time.time() - start_time)
        data_size_histogram.observe(len(json.dumps(data)))

        logger.info(f"Data ingested successfully: ID={ingestion_record['id']}")

        return jsonify({
            'status': 'success',
            'id': ingestion_record['id'],
            'timestamp': ingestion_record['timestamp'],
            }), 201

    except Exception as e:
        logger.error(f"Ingestion error: {str(e)}")
        return jsonify({'error': str(e)}), 500

@app.route('/data/<int:data_id>', methods=['GET'])
def get_data(data_id):
    """Retrieve ingested data by ID"""
    for record in data_store:
        if record['id'] == data_id:
            return jsonify(record), 200
    return jsonify({'error': 'Data not found'}), 404

@app.route('/data', methods=['GET'])
def list_data():
    """List all ingested data"""
    limit = request.args.get('limit', 10, type=int)
    return jsonify({
        'total': len(data_store),
        'data': data_store[-limit:],
        }), 200

@app.route('/metrics', methods=['GET'])
def metrics():
    """Prometheus metrics endpoint"""
    try:
        # Return metrics with proper content type
        return generate_latest(), 200, {'Content-Type': CONTENT_TYPE_LATEST}
    except Exception as e:
        logger.error(f"Metrics error: {str(e)}")
        return "Metrics unavailable", 500
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8002, debug=False)
```

This is a Flask web API that collects and stores incoming data with built-in monitoring and retrieval capabilities. It runs on port 8002 and provides five endpoints: `/health` for service status, `/ingest` for accepting POST requests with data (which gets stored in memory with metadata like timestamp, source IP, and unique ID), `/data/<id>` for retrieving specific records by ID, `/data` for listing recent ingested data with optional limits, and `/metrics` for Prometheus monitoring. When data comes in, the service validates it, adds tracking metadata, simulates processing time, stores it in an in-memory list (which would be a database in production), and tracks metrics like ingestion count, processing duration, and data size. It includes comprehensive error handling, logging, and automatic performance monitoring so you can see how much data is flowing through the system.

In simple terms- It's a data collection API that receives, validates, and stores incoming data while keeping track of what's been processed - like a smart inbox for your data pipeline with built-in monitoring.

Create `services/data-ingestion/requirements.txt`:

```
flask==2.3.3
prometheus-client==0.17.1
gunicorn==21.2.0
```

Create `services/data-ingestion/tests/test_app.py`:

```
import unittest
import json
import sys
import os

# Add the parent directory to Python path so we can import app
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

try:
    from app import app
except ImportError:
    # If direct import fails, try relative import
```

```
    import app as app_module
    app = app_module.app

class TestDataIngestion(unittest.TestCase):
    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True

    def test_health_endpoint(self):
        response = self.app.get('/health')
        self.assertEqual(response.status_code, 200)
        data = json.loads(response.data)
        self.assertEqual(data['status'], 'healthy')

    def test_ingest_endpoint(self):
        test_data = {'sensor': 'temp01', 'value': 25.5}
        response = self.app.post('/ingest',
                                 data=json.dumps(test_data),
                                 content_type='application/json')
        self.assertEqual(response.status_code, 201)
        data = json.loads(response.data)
        self.assertEqual(data['status'], 'success')
        self.assertIn('id', data)

    def test_get_data_endpoint(self):
        # First ingest some data
        test_data = {'sensor': 'temp01', 'value': 25.5}
        response = self.app.post('/ingest',
                                 data=json.dumps(test_data),
                                 content_type='application/json')
        data_id = json.loads(response.data)['id']

        # Then retrieve it
        response = self.app.get(f'/data/{data_id}')
        self.assertEqual(response.status_code, 200)

if __name__ == '__main__':
    unittest.main()
```

This is an automated test suite that validates the data ingestion service works correctly before deployment. It uses Python's unittest framework to create a test client that simulates HTTP requests to the Flask app running on port 8002. The tests cover three main scenarios: verifying the `/health` endpoint returns healthy status, testing that the `/ingest` endpoint successfully

accepts sensor data (like temperature readings) and returns a success response with a unique ID, and ensuring the data retrieval workflow works by first ingesting test data then fetching it back using the returned ID. The test setup handles import complexities to reliably load the Flask app, and it tests the complete data flow from ingestion to retrieval to ensure the service can both store and retrieve data properly.

So,these are automated quality checks that make sure your data ingestion service can properly receive, store, and retrieve data - testing the full lifecycle to catch any bugs before the service goes live.

Step 3: Dockerizing the Microservices

3.1 ML Inference Service Dockerfile

Create `services/ml-inference/Dockerfile`:

```
# Use Python 3.9 slim image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Install system dependencies including curl for health checks
RUN apt-get update && apt-get install -y \
    gcc \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements first for better caching
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Expose port
```

```
EXPOSE 8001

# Run with gunicorn for production
CMD ["gunicorn", "--bind", "0.0.0.0:8001", "--workers", "2",
"--threads", "4", "--timeout", "60", "app:app"]
```

This Dockerfile creates a production-ready container for the ML inference service using Python 3.9 slim base image to keep it lightweight. It sets up the container by installing system dependencies (gcc for compiling packages and curl for health checks), copies and installs Python requirements first for better Docker layer caching, then copies the application code. For security, it creates a non-root user called "appuser" to run the service instead of root, exposes port 8001 for external access, and uses Gunicorn as the production WSGI server with 2 worker processes and 4 threads each for handling concurrent requests. The multi-layered approach optimizes build times by caching dependencies separately from code changes, and the slim image keeps the container size small while including everything needed to run the Flask ML service in production.

3.2 Data Ingestion Service Dockerfile

Create `services/data-ingestion/Dockerfile`:

```
# Use Python 3.9 slim image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Install system dependencies including curl for health checks
RUN apt-get update && apt-get install -y \
    gcc \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements first for better caching
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .
```

```
# Create non-root user
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Expose port
EXPOSE 8002

# Run with gunicorn for production
CMD ["gunicorn", "--bind", "0.0.0.0:8002", "--workers", "2",
"--threads", "4", "--timeout", "60", "app:app"]
```

It's the containerization recipe for your data ingestion service - identical to the ML service setup but listening on port 8002, ensuring both services have consistent, secure, and production-ready deployment packaging.

3.3 Docker Compose for Local Testing

Create `docker-compose.yml` in the root directory:

```yaml
version: '3.8'

services:
  ml-inference:
    build: ./services/ml_inference
    ports:
      - "8001:8001"
    environment:
      - FLASK_ENV=development
    networks:
      - mlops_network

  data-ingestion:
    build: ./services/data_ingestion
    ports:
      - "8002:8002"
    environment:
      - FLASK_ENV=development
    networks:
      - mlops_network

  prometheus:
    image: prom/prometheus:latest
    ports:
```

```
        "9090:9090"
    volumes

./monitoring/prometheus/prometheus.yml /etc/prometheus/prometheus.yml
        prometheus data /prometheus
    command
        '--config.file=/etc/prometheus/prometheus.yml'
        '--storage.tsdb.path=/prometheus'
    networks
        mlops network

  grafana
    image  grafana/grafana latest
    ports
        "3000:3000"
    environment
        GF_SECURITY_ADMIN_PASSWORD=admin
        GF_USERS_ALLOW_SIGN_UP=false
    volumes
        ./monitoring/grafana/provisioning /etc/grafana/provisioning
        ./monitoring/grafana/dashboards /var/lib/grafana/dashboards
        grafana data /var/lib/grafana
    depends_on
        prometheus
    networks
        mlops network

networks
  mlops-network
    driver  bridge

volumes
  prometheus-data
  grafana-data:
```

This Docker Compose file orchestrates all four services into a complete MLOps stack for local development and testing. It defines four services: the ML inference service (built from local code, port 8001), data ingestion service (built from local code, port 8002), Prometheus monitoring (using official image, port 9090 with custom config), and Grafana dashboards (using official image, port 3000 with admin/admin login). All services communicate through a shared "mlops-network" bridge network, enabling internal service discovery by name. The configuration includes persistent volumes for Prometheus metrics and Grafana dashboards so data survives

container restarts, mounts local configuration files for monitoring setup, and sets Grafana to depend on Prometheus to ensure proper startup order.

Step 4: Setting Up Pulumi Infrastructure

4.1 Initialize Pulumi Project

Create `infrastructure/requirements.txt`:

```
pulumi>=3.0.0,<4.0.0
pulumi-aws>=6.0.0,<7.0.0
```

Create `infrastructure/Pulumi.yaml`:

```yaml
name: mlops-pipeline
runtime: python
description: MLOps Pipeline Infrastructure with AWS
template:
  description: A complete MLOps infrastructure setup
  config:
    aws:region:
      description: The AWS region to deploy into
      default: ap-southeast-1
```

It's the configuration file that tells Pulumi "this is a Python-based AWS infrastructure project called mlops-pipeline that should deploy to Singapore by default" - basically the project's identity card for infrastructure deployment.

4.2 Create Infrastructure Code

Create `infrastructure/__main__.py`:

```python
import pulumi
import pulumi_aws as aws
import json
from pulumi import Config, Output
import os

# Use stable suffix instead of timestamp to avoid resource conflicts
unique_suffix = "main"
stack_name = pulumi.get_stack()
```

```
# Get configuration
config = Config()
region = "ap-southeast-1"

# Create key pair from your existing public key
key = aws.ec2.KeyPair("mlops-key",
    key_name=f"mlops-key-{unique_suffix}",
    public_key=os.environ.get("SSH_PUBLIC_KEY", "")    # Will come
from GitHub secret
    tags={"Project": f"mlops-pipeline-{stack_name}",}


# Create VPC
vpc = aws.ec2.Vpc("mlops-vpc",
    cidr_block="10.0.0.0/16",
    enable_dns_hostnames=True,
    enable_dns_support=True,
    tags={
        "Name": f"mlops-vpc-{stack_name}",
        "Project": f"mlops-pipeline-{stack_name}",}


# Create Internet Gateway
igw = aws.ec2.InternetGateway("mlops-igw",
    vpc_id=vpc.id,
    tags={
        "Name": f"mlops-igw-{stack_name}",
        "Project": f"mlops-pipeline-{stack_name}",}


# Create public subnet
public_subnet = aws.ec2.Subnet("mlops-public-subnet",
    vpc_id=vpc.id,
    cidr_block="10.0.1.0/24",
    availability_zone=f"{region}a",
    map_public_ip_on_launch=True,
    tags={
        "Name": f"mlops-public-subnet-{stack_name}",
        "Project": f"mlops-pipeline-{stack_name}",}


# Create route table
```

```
route_table = aws.ec2.RouteTable("mlops-route-table",
    vpc_id=vpc.id,
    routes=[
        aws.ec2.RouteTableRouteArgs(
            cidr_block="0.0.0.0/0",
            gateway_id=igw.id
        )
    ],
    tags={
        "Name": f"mlops-route-table-{stack_name}",
        "Project": f"mlops-pipeline-{stack_name}",
    })


# Associate route table with subnet
route_table_association = aws.ec2.RouteTableAssociation("mlops-rta",
    subnet_id=public_subnet.id,
    route_table_id=route_table.id)


# Create security group
security_group = aws.ec2.SecurityGroup("mlops-sg",
    name=f"mlops-sg-{unique_suffix}",
    vpc_id=vpc.id,
    description="Security group for MLOps services",
    ingress=[
        # SSH
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp",
            from_port=22,
            to_port=22,
            cidr_blocks=["0.0.0.0/0"],
            description="SSH"),

        # ML Inference Service
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp",
            from_port=8001,
            to_port=8001,
            cidr_blocks=["0.0.0.0/0"],
            description="ML Inference Service"),

        # Data Ingestion Service
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp",
```

```
            from_port=8002
            to_port=8002
            cidr_blocks= "0.0.0.0/0"
            description="Data Ingestion Service"
        )

        # Prometheus
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp"
            from_port=9090
            to_port=9090
            cidr_blocks= "0.0.0.0/0"
            description="Prometheus"
        )

        # Grafana
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp"
            from_port=3000
            to_port=3000
            cidr_blocks= "0.0.0.0/0"
            description="Grafana"
        )
    ],

    egress=[
        aws.ec2.SecurityGroupEgressArgs(
            protocol="-1"
            from_port=0
            to_port=0
            cidr_blocks= "0.0.0.0/0"
            description="Allow all outbound traffic"
        )
    ],

    tags={
        "Name": f"mlops-security-group-{stack_name}",
        "Project": f"mlops-pipeline-{stack_name}",
    }
)


# Create ECR repositories with unique names and force delete
ml_inference_repo = aws.ecr.Repository("ml-inference-repo",
    name=f"mlops/ml-inference-{unique_suffix}",
    image_tag_mutability="MUTABLE",
    force_delete=True,  # This allows deletion even with images
    image_scanning_configuration=aws.ecr.RepositoryImageScanningConfigura
tionArgs(
```

```
        scan_on_push=True

    tags=
        "Name"  f"ml-inference-repo- stack_name "
        "Project"  f"mlops-pipeline- stack_name "


data_ingestion_repo = aws ecr Repository "data-ingestion-repo"
    name=f"mlops/data-ingestion- unique_suffix "
    image_tag_mutability="MUTABLE"
    force_delete=True   # This allows deletion even with images

image_scanning_configuration=aws ecr RepositoryImageScanningConfigura
tionArgs
        scan_on_push=True

    tags=
        "Name"  f"data-ingestion-repo- stack_name "
        "Project"  f"mlops-pipeline- stack_name "


# Enhanced user data script (optimized for t2.micro)
user_data = f"""#!/bin/bash
# Update system (but don't upgrade to save time and resources)
apt-get update

# Install Docker (lighter installation for t2.micro)
curl -fsSL https://get.docker.com -o get-docker.sh
sh get-docker.sh
usermod -aG docker ubuntu

# Install Docker Compose (lighter version)
curl -L
"https://github.com/docker/compose/releases/download/v2.24.0/docker-c
ompose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose

# Install AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
apt-get install -y unzip curl jq

# Extract and install AWS CLI
```

```
unzip awscliv2.zip
./aws/install
rm -rf aws awscliv2.zip get-docker.sh

# Configure Docker for t2.micro (limit resources)
mkdir -p /etc/docker
cat > /etc/docker/daemon.json << 'DOCKER_EOF'
{{
  "log-driver": "json-file",
  "log-opts": {{
    "max-size": "10m",
    "max-file": "3"
  }},
  "storage-driver": "overlay2"
}}
DOCKER_EOF

# Start services
systemctl enable docker
systemctl start docker

# Wait for Docker to be ready
timeout=60
while ! docker info > /dev/null 2>&1 && [ $timeout -gt 0 ]; do
    sleep 2
    timeout=$((timeout-2))
done

# Create completion marker - THIS IS IMPORTANT
echo "Setup complete at $(date)" > /home/ubuntu/setup-info.txt
echo "Instance type: t2.micro" >> /home/ubuntu/setup-info.txt
echo "Docker status: $(systemctl is-active docker)" >>
/home/ubuntu/setup-info.txt
chown ubuntu:ubuntu /home/ubuntu/setup-info.txt

# Release package manager locks explicitly
apt-get clean
rm -f /var/lib/apt/lists/lock
rm -f /var/lib/dpkg/lock-frontend
rm -f /var/lib/dpkg/lock

echo "User data script completed successfully" >>
/home/ubuntu/setup-info.txt
"""
```

```
# Create EC2 instance
instance = aws.ec2.Instance("mlops-instance",
    key_name=key.key_name,   # Use the created key pair
    instance_type="t2.micro",   # Changed to t2.micro for free
tier/sandbox
    ami="ami-0df7a207adb9748c7",   # Ubuntu 22.04 LTS in
ap-southeast-1
    subnet_id=public_subnet.id,
    vpc_security_group_ids=[security_group.id],
    user_data=user_data,
    root_block_device=aws.ec2.InstanceRootBlockDeviceArgs(
        volume_type="gp2",   # Changed to gp2 for free tier
compatibility
        volume_size=20,      # Reduced to 20GB for free tier
        delete_on_termination=True
    ),
    tags={
        "Name": f"mlops-instance-{stack_name}",
        "Project": f"mlops-pipeline-{stack_name}",
    },
)

# Create Elastic IP with correct syntax
elastic_ip = aws.ec2.Eip("mlops-eip",
    instance=instance.id,
    domain="vpc",   # Use domain instead of vpc=True
    tags={
        "Name": f"mlops-eip-{stack_name}",
        "Project": f"mlops-pipeline-{stack_name}",
    },
)

# Export outputs
pulumi.export("vpc_id", vpc.id)
pulumi.export("subnet_id", public_subnet.id)
pulumi.export("security_group_id", security_group.id)
pulumi.export("instance_id", instance.id)
pulumi.export("instance_public_ip", elastic_ip.public_ip)
pulumi.export("ml_inference_repo_url",
ml_inference_repo.repository_url)
pulumi.export("data_ingestion_repo_url",
data_ingestion_repo.repository_url)
pulumi.export("grafana_url", Output.concat("http://",
elastic_ip.public_ip, ":3000"))
```

```
pulumi.export("prometheus_url", Output.concat("http://", elastic_ip.public_ip, ":9090"))
pulumi.export("unique_suffix", unique_suffix)
```

This script defines the complete AWS infrastructure for the MLOps pipeline using Infrastructure as Code. It creates a full network stack including a VPC with public subnet, internet gateway, and routing for connectivity, plus security groups that allow access to all service ports (SSH 22, ML service 8001, data service 8002, Prometheus 9090, Grafana 3000). The script provisions two ECR repositories for storing Docker images, creates a t2.micro EC2 instance with an elastic IP for consistent access, and includes a comprehensive user data script that automatically installs Docker, Docker Compose, and AWS CLI when the instance boots up. It's optimized for AWS free tier usage and includes proper resource tagging, cleanup configurations, and exports all important URLs and IDs for use by the CI/CD pipeline.

In simple terms- It's the complete recipe for creating your entire AWS environment - from networking and security to compute and storage - all defined in Python code that Pulumi executes to build your cloud infrastructure automatically.

Step 5: Configuring GitHub Actions

Setting Up GitHub Secrets

What are GitHub Secrets? They're like passwords that your automated pipeline can use, but they're hidden from everyone.

Step-by-step to add secrets:

1. Go to your repository on GitHub.com
2. Click on "Settings" (in the repository menu, not your profile settings)
3. In the left sidebar, scroll down and click "Secrets and variables"
4. Click on "Actions"
5. Click the green "New repository secret" button

Now add these three secrets:

Secret 1: AWS_ACCESS_KEY_ID

- Click "New repository secret"
- Name: `AWS_ACCESS_KEY_ID`
- Secret: Paste your AWS Access Key (from sandbox)

- Click "Add secret"

Secret 2: AWS_SECRET_ACCESS_KEY

- Click "New repository secret" again
- Name: `AWS_SECRET_ACCESS_KEY`
- Secret: Paste your AWS Secret Key (from sandbox)
- Click "Add secret"

Secret 3: EC2_SSH_KEY (We need to create this first)

First, create an SSH key on your computer:

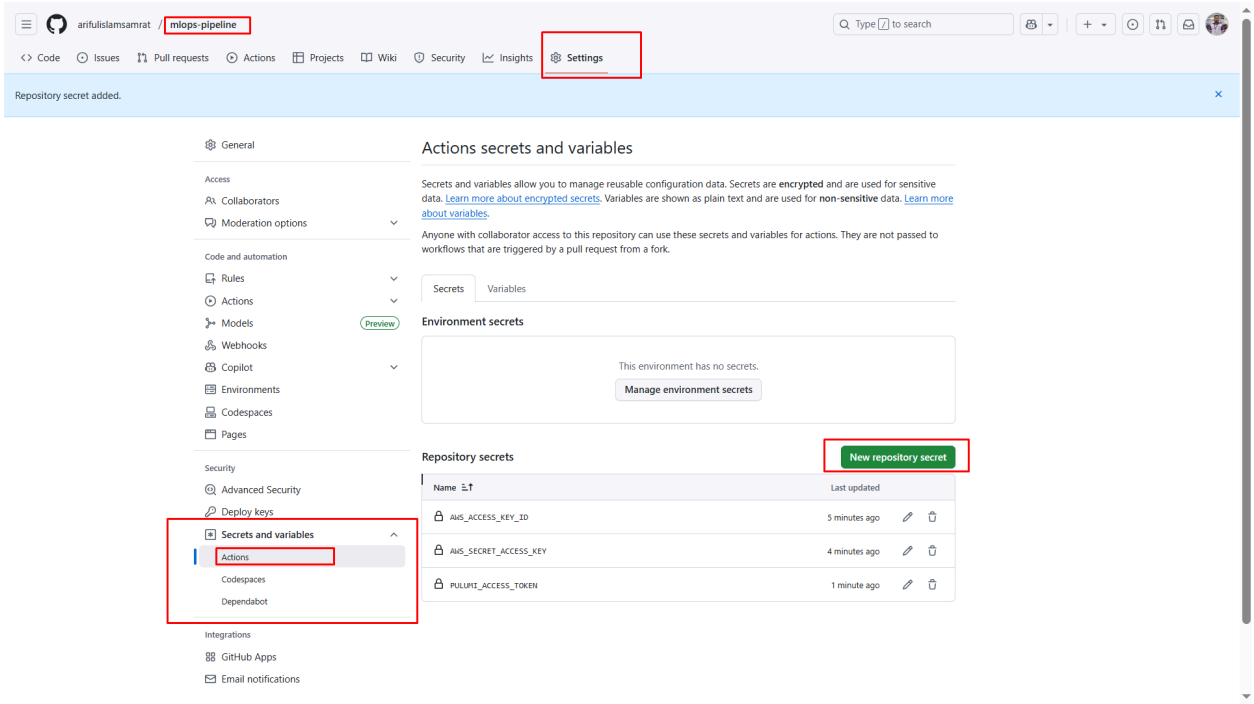
```
# Go to your project directory
cd ~/mlops-pipeline

# Generate SSH key pair (press Enter for all prompts - no passphrase)
ssh-keygen -t rsa -b 4096 -f mlops-key -N ""

# This creates two files:
# - mlops-key (private key - keep this secret!)
# - mlops-key.pub (public key - this can be shared)

# Display your private key
cat mlops-key
```

Copy the ENTIRE output (including `-----BEGIN RSA PRIVATE KEY-----` and `-----END RSA PRIVATE KEY-----`) and add it as a secret:

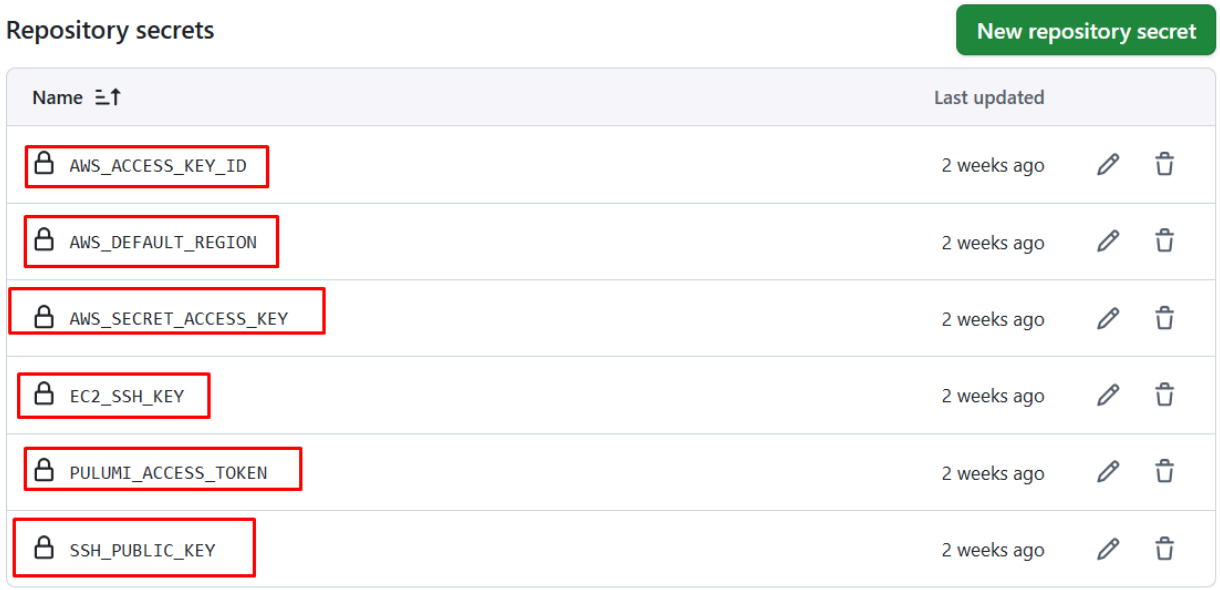
- Click "New repository secret"
- Name: `EC2_SSH_KEY`
- Secret: Paste the entire private key
- Click "Add secret"

Keep the mlops-key.pub file content handy - we'll need it later!

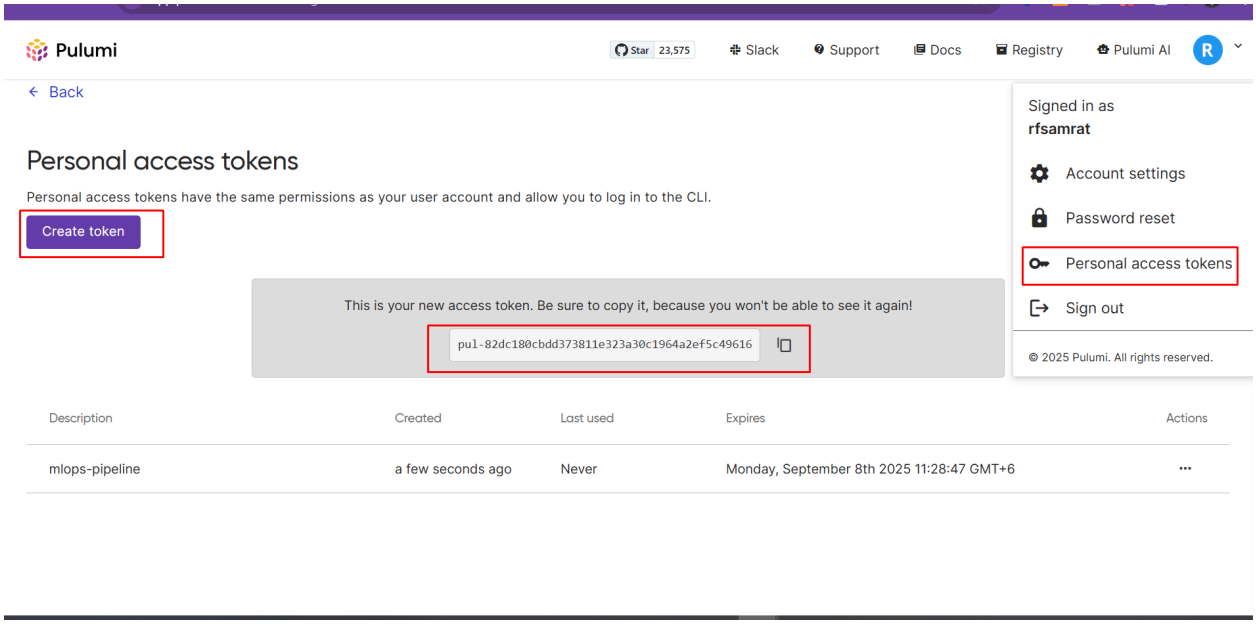
- `AWS_ACCESS_KEY_ID`: Your AWS access key
- `AWS_DEFAULT_REGION`: ap-southeast-1 / your selected region
- `EC2_SSH_KEY`: Your ec2 ssh key
- `AWS_SECRET_ACCESS_KEY`: Your AWS secret key
- `PULUMI_ACCESS_TOKEN`: Create at https://app.pulumi.com/account/tokens
- `SSH_PUBLIC_KEY`: Your SSH public Key



Go to your GitHub repository page then Click on "Settings" (in the repository menu, not your profile) then In the left sidebar, click "Secrets and variables" then Click "Actions" and at end Click the green "New repository secret" button



For pulumi Access token goto pulumi and create project and personal access token



5.2 Create GitHub Actions Workflow

Deployment Flow:

1. Code push triggers GitHub Actions
2. Tests run in parallel for both services
3. Pulumi destroys/recreates infrastructure (clean slate approach)
4. Docker images built and pushed to ECR
5. SSH deployment to EC2 configures and starts all services
6. Health checks validate successful deployment

This architecture emphasizes simplicity and reliability through complete infrastructure refresh on each deployment, comprehensive monitoring, and containerized service isolation.

Create `.github/workflows/deploy.yml`

```yaml
name: Deploy MLOps Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

```
env
  AWS_REGION  ap southeast 1
  PULUMI_STACK  rfsamrat/mlops pipeline/sandbox

jobs
  test
    runs-on  ubuntu latest
    strategy
      matrix
        service   ml inference  data ingestion

    steps
      uses  actions/checkout@v4

      name  Set up Python
      uses  actions/setup python@v4
      with
        python-version  '3.9'

      name  Install dependencies and run tests
      run
        cd services/${{ matrix.service }}
        pip install -r requirements.txt
        pip install pytest

        # Check if tests directory exists
        if    d "tests"  ; then
          echo "Tests directory found  running pytest   "
          python  m pytest tests/  v   tb=short
        else
          echo "Tests directory not found  running basic health
test   "
          python  c "
        import sys
        sys.path.insert(0  '.')
        try
            from app import app
            print('✓ App import successful')
            client = app.test_client()
            response = client.get('/health')
            print(f'✓ Health check   response.status_code ')
            assert response.status_code == 200
            print('✓ Basic health test passed')
        except Exception as e
```

```
            print(f'✗ Test failed   e ')
            sys.exit(1)
        "
      fi

  deploy-infrastructure
    needs  test
    runs-on  ubuntu latest
    if  github.ref == 'refs/heads/main'
    outputs
      instance_ip  $   steps.get outputs.outputs.instance_ip
      ml_inference_repo  $
steps.get outputs.outputs.ml_inference_repo
      data_ingestion_repo  $
steps.get outputs.outputs.data_ingestion_repo
      unique_suffix  $   steps.get outputs.outputs.unique_suffix

    steps
      uses  actions/checkout@v4

      name  Configure AWS credentials
      uses  aws actions/configure aws credentials@v4
      with
        aws-access-key-id  $   secrets.AWS_ACCESS_KEY_ID
        aws-secret-access-key  $   secrets.AWS_SECRET_ACCESS_KEY
        aws-region  $   env.AWS_REGION

      name  Set up Python
      uses  actions/setup python@v4
      with
        python-version  '3.9'

      name  Install Pulumi
      run
        curl -fsSL https://get.pulumi.com | sh
        echo "$HOME/.pulumi/bin" >> $GITHUB_PATH

      name  Complete stack cleanup and fresh start
      env
        PULUMI_ACCESS_TOKEN  $   secrets.PULUMI_ACCESS_TOKEN
      run
        echo "Performing complete stack cleanup..."

        # Clean up any MLOps ECR repositories first
```

```
        REPOS=$(aws ecr describe repositories  query
'repositories  contains(repositoryName  `mlops/`) .repositoryName'
  output text 2 /dev/null    echo "")
        if  !  z "$REPOS"  ; then
          for repo in $REPOS; do
            echo "Force deleting repository  $repo"
            aws ecr delete repository   repository name "$repo"
  force   true
          done
        fi

        cd infrastructure
        pip install  r requirements.txt
        pulumi login

        # Check if stack exists and remove it completely
        if pulumi stack ls   grep  q "$   env.PULUMI_STACK   "; then
          echo "Stack exists  removing completely   "

          # Try to select and destroy (ignore errors)
          pulumi stack select $   env.PULUMI_STACK      true
          pulumi destroy   yes   skip preview   true

          # Force remove the stack entirely
          pulumi stack rm $   env.PULUMI_STACK      yes   force
true
          echo "Stack removed"
        else
          echo "No existing stack found"
        fi

        # Create completely fresh stack
        echo "Creating fresh stack   "
        pulumi stack init $   env.PULUMI_STACK
        echo "Fresh stack created successfully"

      name  Deploy infrastructure
      env
        PULUMI_ACCESS_TOKEN  $   secrets.PULUMI_ACCESS_TOKEN
        SSH_PUBLIC_KEY  $   secrets.SSH_PUBLIC_KEY
      run
        cd infrastructure
        pulumi stack select ${{ env.PULUMI_STACK }}
        pulumi up --yes
```

```
      name: Get deployment outputs
      id: get_outputs
      env:
        PULUMI_ACCESS_TOKEN: ${{ secrets.PULUMI_ACCESS_TOKEN }}
      run: |
        cd infrastructure
        echo "instance_ip=$(pulumi stack output instance_public_ip)"
>> $GITHUB_OUTPUT
        echo "ml_inference_repo=$(pulumi stack output
ml_inference_repo_url)" >> $GITHUB_OUTPUT
        echo "data_ingestion_repo=$(pulumi stack output
data_ingestion_repo_url)" >> $GITHUB_OUTPUT
        echo "unique_suffix=$(pulumi stack output unique_suffix)" >>
$GITHUB_OUTPUT

  build-and-push:
    needs: deploy-infrastructure
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    strategy:
      matrix:
        service: [ml-inference, data-ingestion]

    steps:
      uses: actions/checkout@v4

      name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: ${{ env.AWS_REGION }}

      name: Login to Amazon ECR
      id: login-ecr
      uses: aws-actions/amazon-ecr-login@v2

      name: Get Pulumi outputs
      id: pulumi-outputs
      run: |
        if [ "${{ matrix.service }}" == "ml-inference" ]; then
          echo "repository_url=${{
needs.deploy-infrastructure.outputs.ml_inference_repo }}" >>
$GITHUB_OUTPUT
        else
```

```
          echo "repository_url=${{
needs.deploy-infrastructure.outputs.data_ingestion_repo }}" >>
$GITHUB_OUTPUT
        fi

      name: Build, tag, and push image to Amazon ECR
      env:
        ECR_REPOSITORY: ${{
steps.pulumi_outputs.outputs.repository_url }}
        IMAGE_TAG: ${{ github.sha }}
      run: |
        cd services/${{ matrix.service }}

        # Debug: List files to ensure we're in the right place
        echo "Current directory contents:"
        ls -la

        # Build the Docker image
        docker build -t $ECR_REPOSITORY:$IMAGE_TAG .
        docker tag $ECR_REPOSITORY:$IMAGE_TAG $ECR_REPOSITORY:latest

        # Push images
        docker push $ECR_REPOSITORY:$IMAGE_TAG
        docker push $ECR_REPOSITORY:latest

        echo "Successfully built and pushed ${{ matrix.service }}"

  deploy-services:
    needs: [deploy-infrastructure, build-and-push]
    runs-on: ubuntu-latest
    if: ${{ github.ref == 'refs/heads/main' }}

    steps:
      uses: actions/checkout@v4

      name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: ${{ env.AWS_REGION }}

      name: Get instance details
      id: get-instance
      run:
```

```
        echo "instance_ip=${{
needs.deploy-infrastructure.outputs.instance_ip }}" >> $GITHUB_OUTPUT
        echo "unique_suffix=${{
needs.deploy-infrastructure.outputs.unique_suffix }}" >>
$GITHUB_OUTPUT

      name  Wait for EC2 instance to be ready
      run
        echo "Waiting for EC2 instance to be fully ready..."
        INSTANCE_IP="${{ steps.get-instance.outputs.instance_ip }}"

        # Wait for SSH to be available
        timeout=300
        while ! nc  z $INSTANCE_IP 22; do
          echo "Waiting for SSH on $INSTANCE_IP   "
          sleep 10
          timeout=$((timeout 10))
          if   $timeout  le 0  ; then
            echo "SSH timeout reached"
            break
          fi
        done

        echo "SSH is available on $INSTANCE_IP"
        sleep 60  # Longer buffer time for user_data to complete

      name  Deploy services to EC2
      uses  appleboy/ssh action@v1.0.3
      env
        AWS_ACCESS_KEY_ID  $   secrets.AWS_ACCESS_KEY_ID
        AWS_SECRET_ACCESS_KEY  $   secrets.AWS_SECRET_ACCESS_KEY
        AWS_DEFAULT_REGION  ap southeast 1
        UNIQUE_SUFFIX  $   steps.get instance.outputs.unique_suffix

      with
        host  $   steps.get instance.outputs.instance_ip
        username  ubuntu
        key  $   secrets.EC2_SSH_KEY
        timeout  30m
        command_timeout  30m
        envs
AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_DEFAULT_REGION UNIQUE_SUF
FIX
        script
          echo "=== Starting EC2 Service Deployment ==="
```

```
          # Wait for setup completion with longer timeout
          echo "Waiting for EC2 initialization to complete...."
          timeout=1200  # 20 minutes
          while ! -f /home/ubuntu/setup_info.txt && [[ $timeout -gt 0 ]]; do
            echo "Waiting for setup completion... ($timeout seconds remaining)"
            sleep 20
            timeout=$((timeout-20))
          done

          if ! -f /home/ubuntu/setup_info.txt ; then
            echo "⚠️ Setup timeout - proceeding with manual setup..."

            # Manual Docker installation (no apt-get)
            if ! command -v docker &> /dev/null; then
              echo "Installing Docker manually..."
              curl -fsSL https://get.docker.com -o get-docker.sh
              sudo sh get-docker.sh
              sudo usermod -aG docker ubuntu
              sudo systemctl start docker
              sudo systemctl enable docker
            fi

            # Manual Docker Compose installation
            if ! command -v docker-compose &> /dev/null; then
              echo "Installing Docker Compose..."
              sudo curl -L "https://github.com/docker/compose/releases/download/v2.24.0/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
              sudo chmod +x /usr/local/bin/docker-compose
            fi

            # Manual AWS CLI installation (no apt-get)
            if ! command -v aws &> /dev/null; then
              echo "Installing AWS CLI..."
              curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
              unzip awscliv2.zip
              sudo ./aws/install
            fi
          else
```

```
        echo "✅ EC2 setup completed successfully!"
        cat /home/ubuntu/setup_info.txt
    fi

    # Ensure Docker is running
    echo "Starting Docker service...."
    sudo systemctl start docker
    sudo systemctl enable docker

    # Wait for Docker to be ready
    echo "Waiting for Docker to be ready...."
    timeout 300 bash -c 'while ! sudo docker info >/dev/null
2 &1; do echo "Waiting for Docker..."; sleep 5; done'

    if sudo docker info >/dev/null 2 &1; then
        echo "✅ Docker is ready!"
    else
        echo "❌ Docker failed to start"
        sudo systemctl status docker
        exit 1
    fi

    # Configure AWS CLI with credentials from environment
    echo "Configuring AWS CLI with provided credentials...."
    aws configure set aws_access_key_id "$AWS_ACCESS_KEY_ID"
    aws configure set aws_secret_access_key
"$AWS_SECRET_ACCESS_KEY"
    aws configure set default.region ap-southeast-1

    # Test AWS credentials
    echo "Testing AWS credentials...."
    if aws sts get-caller-identity; then
        echo "✅ AWS credentials working"
    else
        echo "❌ AWS credentials not working"
        exit 1
    fi

    # Login to ECR
    echo "Logging into ECR...."
    aws ecr get-login-password --region ap-southeast-1 | sudo
docker login --username AWS --password-stdin $(aws sts
get-caller-identity --query Account --output
text).dkr.ecr.ap-southeast-1.amazonaws.com
```

```
# Create monitoring directories
echo "Setting up monitoring directories   "
mkdir  p /home/ubuntu/monitoring/prometheus
mkdir  p
/home/ubuntu/monitoring/grafana/provisioning/datasources
mkdir  p
/home/ubuntu/monitoring/grafana/provisioning/dashboards
mkdir  p /home/ubuntu/monitoring/grafana/dashboards

# Create Prometheus config
cat   /home/ubuntu/monitoring/prometheus/prometheus.yml <<
'PROMETHEUS_EOF'
global
  scrape_interval  15s
  evaluation_interval  15s

scrape_configs
    job_name  'prometheus'
    static_configs
        targets   'localhost:9090'

    job_name  'ml-inference'
    static_configs
        targets   'ml-inference:8001'
    metrics_path  '/metrics'
    scrape_interval  10s

    job_name  'data-ingestion'
    static_configs
        targets   'data-ingestion:8002'
    metrics_path  '/metrics'
    scrape_interval  10s
PROMETHEUS_EOF

# Create Grafana datasource config
cat
/home/ubuntu/monitoring/grafana/provisioning/datasources/prometheus.y
ml << 'DATASOURCE_EOF'
apiVersion  1
datasources
    name  Prometheus
    type  prometheus
    access  proxy
    url  http //prometheus 9090
    isDefault  true
```

```
            editable: false
        DATASOURCE_EOF

        # Create Grafana dashboard provisioning config
        cat > /home/ubuntu/monitoring/grafana/provisioning/dashboards/dashboard.yml << 'DASHBOARD_EOF'
        apiVersion: 1
        providers:
            name: 'MLOps Dashboards'
            orgId: 1
            folder: ''
            type: file
            disableDeletion: false
            updateIntervalSeconds: 10
            allowUiUpdates: true
            options:
              path: /var/lib/grafana/dashboards
              foldersFromFilesStructure: true
        DASHBOARD_EOF

        # Get ECR repository URLs
        echo "Using unique suffix: $UNIQUE_SUFFIX"

        ML_INFERENCE_REPO=$(aws ecr describe-repositories --repository-names "mlops/ml-inference-$UNIQUE_SUFFIX" --query 'repositories[0].repositoryUri' --output text 2>/dev/null || echo "")
        DATA_INGESTION_REPO=$(aws ecr describe-repositories --repository-names "mlops/data-ingestion-$UNIQUE_SUFFIX" --query 'repositories[0].repositoryUri' --output text 2>/dev/null || echo "")

        if [ -z "$ML_INFERENCE_REPO" ] || [ -z "$DATA_INGESTION_REPO" ]; then
          echo "❌ Error: Could not retrieve ECR repository URLs"
          echo "Available repositories:"
          aws ecr describe-repositories --query 'repositories[*].repositoryName' --output text
          exit 1
        fi

        echo "✅ Repository URLs retrieved:"
        echo "  ML Inference: $ML_INFERENCE_REPO"
        echo "  Data Ingestion: $DATA_INGESTION_REPO"

        # Create docker-compose.yml
```

```
          cat   /home/ubuntu/docker compose.yml << COMPOSE_EOF
          version  '3.8'
          services
            ml-inference
              image  $ ML_INFERENCE_REPO  latest
              ports
                  "8001:8001"
              restart  always
              networks
                  mlops network
              healthcheck
                test   "CMD"  "curl"  "-f"
"http://localhost:8001/health"
                interval  30s
                timeout  10s
                retries  3
                start_period  90s

            data-ingestion
              image  $ DATA_INGESTION_REPO  latest
              ports
                  "8002:8002"
              restart  always
              networks
                  mlops network
              healthcheck
                test   "CMD"  "curl"  "-f"
"http://localhost:8002/health"
                interval  30s
                timeout  10s
                retries  3
                start_period  90s

            prometheus
              image  prom/prometheus latest
              ports
                  "9090:9090"
              volumes

./monitoring/prometheus/prometheus.yml /etc/prometheus/prometheus.yml
                  prometheus data /prometheus
              command
                  '--config.file=/etc/prometheus/prometheus.yml'
                  '--storage.tsdb.path=/prometheus'
```

```
'--web.console.libraries=/etc/prometheus/console_libraries'
                '--web.console.templates=/etc/prometheus/consoles'
                '--storage.tsdb.retention.time=200h'
                '--web.enable-lifecycle'
            restart: always
            networks:
                - mlops-network

          grafana:
            image: grafana/grafana:latest
            ports:
                - "3000:3000"
            environment:
                - GF_SECURITY_ADMIN_PASSWORD=admin
                - GF_USERS_ALLOW_SIGN_UP=false
            volumes:
                - ./monitoring/grafana/provisioning:/etc/grafana/provisioning:ro
                - ./monitoring/grafana/dashboards:/var/lib/grafana/dashboards:ro
                - grafana-data:/var/lib/grafana
            depends_on:
                - prometheus
            restart: always
            networks:
                - mlops-network

        networks:
          mlops-network:
            driver: bridge

        volumes:
          prometheus-data:
          grafana-data:
        COMPOSE_EOF

        # Pull latest images
        echo "📥 Pulling latest Docker images..."
        sudo docker pull $ML_INFERENCE_REPO:latest || exit 1
        sudo docker pull $DATA_INGESTION_REPO:latest || exit 1

        # Deploy services
        echo "🚀 Deploying MLOps services..."
        cd /home/ubuntu
```

```
          sudo docker compose down  true
          sudo docker compose up  d

          # Wait for services to start
          echo "⏳ Waiting for services to initialize  "
          sleep 120

          # Check service health
          echo "🔍 Checking service health  "
          success=false
          for i in  1..20 ; do
            echo "Health check attempt $i/20  "

            ML_HEALTH=$(curl  s  o /dev/null  w "% http_code "
http //localhost 8001/health 2 /dev/null  echo "000")
            DATA_HEALTH=$(curl  s  o /dev/null  w "% http_code "
http //localhost 8002/health 2 /dev/null  echo "000")

            echo "  🤖 ML Inference  $ML_HEALTH"
            echo "  📊 Data Ingestion  $DATA_HEALTH"

            if  "$ML_HEALTH" = "200"  &&  "$DATA_HEALTH" = "200"
; then
              echo "✅ All core services are healthy!"
              success=true
              break
            else
              echo "⏳ Services starting up  waiting 25 seconds  "
              sleep 25
            fi
          done

          if  "$success" = true  ; then
            echo ""
            echo "🎉 MLOps Pipeline Deployment Successful!"
            echo "==========================================="

            PUBLIC_IP=$(curl  s ifconfig.me 2 /dev/null  echo
"UNKNOWN")
            echo "🌐 Public IP  $PUBLIC_IP"
            echo ""
            echo "📊 Service Access URLs "
            echo "  🤖 ML Inference Service  http //$PUBLIC_IP 8001"
            echo "  📈 Data Ingestion Service
http //$PUBLIC_IP 8002"
```

```
            echo "  📊 Prometheus Monitoring
http //$PUBLIC_IP 9090"
            echo "  📈 Grafana Dashboard       http //$PUBLIC_IP 3000
(admin/admin)"
            echo ""
            echo "🔗 Test your services "
            echo "  curl http //$PUBLIC_IP 8001/health"
            echo "  curl http //$PUBLIC_IP 8002/health"
            echo ""
            echo "✅ Deployment completed successfully!"
          else
            echo "❌ Service health check failed after 20 attempts"
            echo ""
            echo "🔍 Container status "
            sudo docker compose ps
            echo ""
            echo "📋 Recent container logs "
            sudo docker compose logs   tail=15
            exit 1
          fi
```

This GitHub Actions workflow automatically deploys a complete MLOps pipeline to AWS whenever code is pushed to the main branch. The pipeline first runs tests on two ML services (ml-inference and data-ingestion), then uses Pulumi to set up AWS infrastructure including ECR repositories and an EC2 instance. Next, it builds Docker images for both services and pushes them to ECR. Finally, it connects to the EC2 instance via SSH and deploys four containerized services using Docker Compose: the ML inference service (port 8001), data ingestion service (port 8002), Prometheus for monitoring (port 9090), and Grafana for dashboards (port 3000). This demonstrates a complete CI/CD pipeline for machine learning applications, automatically handling everything from testing to production deployment with built-in monitoring and observability.

Step 6: Monitoring Configuration - Complete Files

6.1 Prometheus Configuration

Create `monitoring/prometheus/prometheus.yml`:

```
global
  scrape_interval  15s
```

```
  evaluation_interval  15s

alerting
  alertmanagers
      static_configs
          targets

rule_files
    "alerts.yml"

scrape_configs
    job_name  'prometheus'
    static_configs
        targets   'localhost:9090'

    job_name  'ml-inference'
    static_configs
        targets   'ml-inference:8001'
    metrics_path  '/metrics'
    scrape_interval  10s

    job_name  'data-ingestion'
    static_configs
        targets   'data-ingestion:8002'
    metrics_path  '/metrics'
    scrape_interval  10s
EOF
```

This Prometheus configuration file defines how the monitoring system collects metrics from your MLOps services. The global settings establish a 15-second collection interval for all metrics and evaluations. It configures three monitoring targets: Prometheus itself on localhost:9090, the ML inference service on port 8001, and the data ingestion service on port 8002, with the latter two being scraped every 10 seconds from their `/metrics` endpoints. The configuration also includes placeholders for alertmanagers (currently empty) and references an "alerts.yml" file for defining monitoring rules and alerts, making this a complete monitoring setup that tracks the health and performance of all services in the MLOps pipeline.

6.2 Prometheus Alert Rules

Create `monitoring/prometheus/alerts.yml`:

```
groups
```

```
    name  mlops_alerts
    interval  30s
    rules
        alert  ServiceDown
        expr  up == 0
        for  1m
        labels
          severity  critical
        annotations
          summary  "Service {{ $labels.job }} is down"
          description  "{{ $labels.job }} has been down for more than
1 minute"

        alert  HighResponseTime
        expr  histogram_quantile(0.95
rate(data_ingestion_duration_seconds_bucket 5m ))   1
        for  5m
        labels
          severity  warning
        annotations
          summary  "High response time on {{ $labels.job }}"
          description  "95th percentile response time is {{ $value }}
seconds"

        alert  LowIngestionRate
        expr  rate(data_ingestion_total 5m ) < 0.01
        for  10m
        labels
          severity  info
        annotations
          summary  "Low data ingestion rate"
          description  "Data ingestion service is receiving less than
0.01 requests per second"

        alert  LargeDataSize
        expr  histogram_quantile(0.95
rate(data_size_bytes_bucket 5m ))   10485760
        for  5m
        labels
          severity  warning
        annotations
          summary  "Large data payloads detected"
          description  "95th percentile data size is {{ $value }}
bytes (>10MB)"
```

```
        alert: MLInferenceServiceDown
        expr: up{job="ml-inference"} == 0
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "ML Inference service is down"
          description: "ML Inference service has been down for more
than 2 minutes"

        alert: DataIngestionServiceDown
        expr: up{job="data-ingestion"} == 0
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "Data Ingestion service is down"
          description: "Data Ingestion service has been down for more
than 2 minutes"
```

This Prometheus alerting rules configuration defines monitoring alerts for the MLOps pipeline services. The configuration creates six different alerts within the "mlops_alerts" group that are evaluated every 30 seconds: ServiceDown triggers when any service is unreachable for over 1 minute, HighResponseTime warns when the data ingestion service's 95th percentile response time exceeds 1 second for 5 minutes, LowIngestionRate alerts when data ingestion drops below 0.01 requests per second for 10 minutes, LargeDataSize warns when data payloads exceed 10MB, and two specific service alerts (MLInferenceServiceDown and DataIngestionServiceDown) that trigger when either the ML inference or data ingestion services are down for more than 2 minutes. Each alert includes severity levels (critical, warning, info) and descriptive messages to help operators quickly understand and respond to issues in the MLOps system.

6.3 Grafana Datasource Configuration

Create `monitoring/grafana/provisioning/datasources/prometheus.yml`:

```
apiVersion: 1

datasources:
  - name: Prometheus
```

```
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: false
    jsonData:
      timeInterval: "15s"
      queryTimeout: "60s"
      httpMethod: "POST"
EOF
```

It connects Grafana to the Prometheus monitoring system. The configuration defines Prometheus as the default and primary data source for Grafana dashboards, accessible via the internal Docker network URL "http://prometheus:9090". It sets up the connection with a 15-second data collection interval, 60-second query timeout, and uses POST method for HTTP requests, while making the datasource non-editable to prevent accidental configuration changes. This allows Grafana to automatically pull metrics from Prometheus and display them in monitoring dashboards for visualizing the MLOps pipeline's performance and health status.

6.4 Grafana Dashboard Provisioning

Create `monitoring/grafana/provisioning/dashboards/dashboard.yml`:

```
apiVersion: 1

providers:
    name: 'MLOps Dashboards'
    orgId: 1
    folder: ''
    type: file
    disableDeletion: false
    updateIntervalSeconds: 10
    allowUiUpdates: true
    options:
      path: /var/lib/grafana/dashboards
      foldersFromFilesStructure: true
EOF
```

It automatically loads and manages dashboards for the MLOps system. The configuration tells Grafana to scan the `/var/lib/grafana/dashboards` directory every 10 seconds for

dashboard files and automatically import them into the "MLOps Dashboards" provider. It allows users to modify dashboards through the UI (`allowUiUpdates: true`), prevents accidental deletion of provisioned dashboards, and organizes dashboards using the folder structure found in the file system. This enables automatic deployment and updates of monitoring dashboards whenever new dashboard files are added to the specified directory, ensuring the MLOps monitoring setup stays current without manual intervention.

6.5 Complete Grafana Dashboard

Create `monitoring/grafana/dashboards/mlops-dashboard.json`:

```
  "annotations":
    "list":

        "builtIn": 1
        "datasource":
          "type": "prometheus"
          "uid": "${DS_PROMETHEUS}"

        "enable": true
        "hide": true
        "iconColor": "rgba(0, 211, 255, 1)"
        "name": "Annotations & Alerts"
        "type": "dashboard"



  "editable": true
  "fiscalYearStartMonth": 0
  "graphTooltip": 0
  "id": null
  "links":
  "liveNow": false
  "panels":

      "datasource":
        "type": "prometheus"
        "uid": "${DS_PROMETHEUS}"

      "fieldConfig":
        "defaults":
          "color":
```

```
        "mode": "thresholds"

      "mappings":

          "options":
            "0":
              "color": "red"
              "index": 1
              "text": "DOWN"

            "1":
              "color": "green"
              "index": 0
              "text": "UP"


          "type": "value"


      "thresholds":
        "mode": "absolute"
        "steps":

            "color": "red"
            "value": null


            "color": "green"
            "value": 1



      "unit": "short"

    "overrides":

  "gridPos":
    "h": 8
    "w": 12
    "x": 0
    "y": 0

  "id": 1
  "options":
    "colorMode": "background"
```

```
    "graphMode": "none"
    "justifyMode": "center"
    "orientation": "auto"
    "reduceOptions":
      "values": false
      "calcs":
        "lastNotNull"

      "fields": ""

    "text":
    "textMode": "auto"

  "pluginVersion": "9.5.2"
  "targets":

      "datasource":
        "type": "prometheus"
        "uid": "${DS_PROMETHEUS}"

      "editorMode": "code"
      "expr": "up{job=\"ml-inference\"}"
      "legendFormat": "ML Inference"
      "range": true
      "refId": "A"


      "datasource":
        "type": "prometheus"
        "uid": "${DS_PROMETHEUS}"

      "editorMode": "code"
      "expr": "up{job=\"data-ingestion\"}"
      "legendFormat": "Data Ingestion"
      "range": true
      "refId": "B"


  "title": "Service Health Status"
  "type": "stat"


  "datasource":
    "type": "prometheus"
    "uid": "${DS_PROMETHEUS}"
```

```
"fieldConfig":
  "defaults":
    "color":
      "mode": "palette-classic"

    "custom":
      "axisCenteredZero": false
      "axisColorMode": "text"
      "axisLabel": ""
      "axisPlacement": "auto"
      "barAlignment": 0
      "drawStyle": "line"
      "fillOpacity": 10
      "gradientMode": "none"
      "hideFrom":
        "tooltip": false
        "viz": false
        "legend": false

      "insertNulls": false
      "lineInterpolation": "linear"
      "lineWidth": 1
      "pointSize": 5
      "scaleDistribution":
        "type": "linear"

      "showPoints": "never"
      "spanNulls": false
      "stacking":
        "group": "A"
        "mode": "none"

      "thresholdsStyle":
        "mode": "off"


    "mappings":
    "thresholds":
      "mode": "absolute"
      "steps":

          "color": "green"
          "value": null
```

```
          "color": "red"
          "value": 80


    "unit": "reqps"

  "overrides":

"gridPos":
  "h": 8
  "w": 12
  "x": 12
  "y": 0

"id": 2
"options":
  "legend":
    "calcs":  "mean"  "lastNotNull"
    "displayMode": "table"
    "placement": "bottom"
    "showLegend": true

  "tooltip":
    "mode": "single"
    "sort": "none"


"pluginVersion": "9.5.2"
"targets":

    "datasource":
      "type": "prometheus"
      "uid": "${DS_PROMETHEUS}"

    "editorMode": "code"
    "expr": "rate(data_ingestion_total[5m])"
    "legendFormat": "Data Ingestions per second"
    "range": true
    "refId": "A"


"title": "Data Ingestion Rate"
"type": "timeseries"
```

```
"datasource":
  "type": "prometheus"
  "uid": "${DS_PROMETHEUS}"

"fieldConfig":
  "defaults":
    "color":
      "mode": "palette-classic"

    "custom":
      "axisCenteredZero": false
      "axisColorMode": "text"
      "axisLabel": ""
      "axisPlacement": "auto"
      "barAlignment": 0
      "drawStyle": "line"
      "fillOpacity": 10
      "gradientMode": "none"
      "hideFrom":
        "tooltip": false
        "viz": false
        "legend": false

      "insertNulls": false
      "lineInterpolation": "linear"
      "lineWidth": 1
      "pointSize": 5
      "scaleDistribution":
        "type": "linear"

      "showPoints": "never"
      "spanNulls": false
      "stacking":
        "group": "A"
        "mode": "none"

      "thresholdsStyle":
        "mode": "off"


    "mappings":
    "thresholds":
      "mode": "absolute"
```

```
            "steps":

                "color": "green"
                "value": null


                "color": "red"
                "value": 80



          "unit": "s"

        "overrides":

      "gridPos":
        "h": 8
        "w": 12
        "x": 0
        "y": 8

      "id": 3
      "options":
        "legend":
          "calcs":  "mean"  "max"
          "displayMode": "table"
          "placement": "bottom"
          "showLegend": true

        "tooltip":
          "mode": "single"
          "sort": "none"


      "pluginVersion": "9.5.2"
      "targets":

          "datasource":
            "type": "prometheus"
            "uid": "${DS_PROMETHEUS}"

          "editorMode": "code"
          "expr": "histogram_quantile(0.95,
rate(data_ingestion_duration_seconds_bucket[5m]))"
          "legendFormat": "95th percentile"
```

```
          "range": true
          "refId": "A"


          "datasource":
            "type": "prometheus"
            "uid": "${DS_PROMETHEUS}"

          "editorMode": "code"
          "expr": "histogram_quantile(0.99,
rate(data_ingestion_duration_seconds_bucket[5m]))"
          "legendFormat": "99th percentile"
          "range": true
          "refId": "B"


      "title": "Data Ingestion Latency"
      "type": "timeseries"


      "datasource":
        "type": "prometheus"
        "uid": "${DS_PROMETHEUS}"

      "fieldConfig":
        "defaults":
          "color":
            "mode": "palette-classic"

          "custom":
            "axisCenteredZero": false
            "axisColorMode": "text"
            "axisLabel": ""
            "axisPlacement": "auto"
            "barAlignment": 0
            "drawStyle": "line"
            "fillOpacity": 10
            "gradientMode": "none"
            "hideFrom":
              "tooltip": false
              "viz": false
              "legend": false

            "insertNulls": false
            "lineInterpolation": "linear"
```

```
      "lineWidth": 1
      "pointSize": 5
      "scaleDistribution":
        "type": "linear"

      "showPoints": "never"
      "spanNulls": false
      "stacking":
        "group": "A"
        "mode": "none"

      "thresholdsStyle":
        "mode": "off"


    "mappings":
    "thresholds":
      "mode": "absolute"
      "steps":

          "color": "green"
          "value": null


          "color": "red"
          "value": 80



    "unit": "bytes"

  "overrides":

"gridPos":
  "h": 8
  "w": 12
  "x": 12
  "y": 8

"id": 4
"options":
  "legend":
    "calcs":  "mean"  "lastNotNull"
    "displayMode": "table"
    "placement": "bottom"
```

```
          "showLegend": true

        "tooltip":
          "mode": "single"
          "sort": "none"


      "pluginVersion": "9.5.2"
      "targets":

          "datasource":
            "type": "prometheus"
            "uid": "${DS_PROMETHEUS}"

          "editorMode": "code"
          "expr": "histogram_quantile(0.95,
rate(data_size_bytes_bucket[5m]))"
          "legendFormat": "Data Size 95th percentile"
          "range": true
          "refId": "A"


      "title": "Data Size Distribution"
      "type": "timeseries"


      "datasource":
        "type": "prometheus"
        "uid": "${DS_PROMETHEUS}"

      "fieldConfig":
        "defaults":
          "color":
            "mode": "stat"

          "mappings":
          "thresholds":
            "mode": "absolute"
            "steps":

                "color": "green"
                "value": null


                "color": "red"
```

```
          "value": 80


    "unit": "short"

  "overrides":

"gridPos":
  "h": 4
  "w": 8
  "x": 0
  "y": 16

"id": 5
"options":
  "colorMode": "value"
  "graphMode": "area"
  "justifyMode": "auto"
  "orientation": "auto"
  "reduceOptions":
    "values": false
    "calcs":
      "lastNotNull"

    "fields": ""

  "text":
  "textMode": "auto"

"pluginVersion": "9.5.2"
"targets":

    "datasource":
      "type": "prometheus"
      "uid": "${DS_PROMETHEUS}"

    "editorMode": "code"
    "expr": "sum(data_ingestion_total)"
    "legendFormat": "Total Data Ingestions"
    "range": false
    "refId": "A"


"title": "Total Data Ingestions"
```

```
"type": "stat"


"datasource":
  "type": "prometheus"
  "uid": "${DS_PROMETHEUS}"

"fieldConfig":
  "defaults":
    "color":
      "mode": "thresholds"

    "mappings":
    "thresholds":
      "mode": "absolute"
      "steps":

          "color": "green"
          "value": null


          "color": "yellow"
          "value": 0.1


          "color": "red"
          "value": 0.5



    "unit": "s"

  "overrides":

"gridPos":
  "h": 4
  "w": 8
  "x": 8
  "y": 16

"id": 6
"options":
  "colorMode": "background"
  "graphMode": "area"
  "justifyMode": "auto"
```

```
        "orientation": "auto"
        "reduceOptions":
          "values": false
          "calcs":
            "lastNotNull"

          "fields": ""

        "text":
        "textMode": "auto"

      "pluginVersion": "9.5.2"
      "targets":

          "datasource":
            "type": "prometheus"
            "uid": "${DS_PROMETHEUS}"

          "editorMode": "code"
          "expr": "avg(rate(data_ingestion_duration_seconds_sum[5m])
/ rate(data_ingestion_duration_seconds_count[5m]))"
          "legendFormat": "Avg Response Time"
          "range": false
          "refId": "A"


      "title": "Average Response Time"
      "type": "stat"


      "datasource":
        "type": "prometheus"
        "uid": "${DS_PROMETHEUS}"

      "fieldConfig":
        "defaults":
          "color":
            "mode": "thresholds"

          "mappings":
          "thresholds":
            "mode": "absolute"
            "steps":

                "color": "red"
```

```
          "value": null

          "color": "yellow"
          "value": 1

          "color": "green"
          "value": 2

    "unit": "short"

  "overrides":

"gridPos":
  "h": 4
  "w": 8
  "x": 16
  "y": 16

"id": 7
"options":
  "colorMode": "background"
  "graphMode": "none"
  "justifyMode": "auto"
  "orientation": "auto"
  "reduceOptions":
    "values": false
    "calcs":
      "lastNotNull"

    "fields": ""

  "text":
  "textMode": "auto"

"pluginVersion": "9.5.2"
"targets":

    "datasource":
      "type": "prometheus"
      "uid": "${DS_PROMETHEUS}"
```

```
          "editorMode": "code"
          "expr": "sum(up)"
          "legendFormat": "Services Up"
          "range": false
          "refId": "A"


      "title": "Services Online"
      "type": "stat"


  "refresh": "10s"
  "schemaVersion": 38
  "style": "dark"
  "tags":  "mlops"  "monitoring"
  "templating":
    "list":

        "current":
          "selected": false
          "text": "Prometheus"
          "value": "prometheus"

        "hide": 0
        "includeAll": false
        "label": "Datasource"
        "multi": false
        "name": "DS_PROMETHEUS"
        "options":
        "query": "prometheus"
        "refresh": 1
        "regex": ""
        "skipUrlSync": false
        "type": "datasource"



  "time":
    "from": "now-1h"
    "to": "now"

  "timepicker":
    "refresh_intervals":  "5s"  "10s"  "30s"  "1m"  "5m"  "15m"
"30m"  "1h"  "2h"  "1d"
```

```
  "timezone": ""
  "title": "MLOps Pipeline Dashboard"
  "uid": "mlops-main"
  "version": 1
  "weekStart": ""
}
```

It creates a comprehensive real-time monitoring interface for the MLOps pipeline with seven interconnected visualization panels. The dashboard features a Service Health Status panel that displays UP/DOWN status for both ML inference and data ingestion services using color-coded stat visualizations (green for UP, red for DOWN) with background color mapping for immediate visual identification. A Data Ingestion Rate time-series chart tracks the rate of data ingestions per second over time using Prometheus rate calculations, helping identify traffic patterns and potential bottlenecks. The Data Ingestion Latency panel monitors system performance by displaying both 95th and 99th percentile response times as line graphs, crucial for understanding service responsiveness under load. A Data Size Distribution chart tracks the 95th percentile of incoming data payload sizes in bytes, helping detect unusually large data transfers that might impact performance. Three summary statistic panels provide quick insights: Total Data Ingestions shows cumulative processing count, Average Response Time displays current system responsiveness with color-coded thresholds (green under 0.1s, yellow up to 0.5s, red above), and Services Online counts the number of active services with color coding (red for 0, yellow for 1, green for 2+ services). The dashboard auto-refreshes every 10 seconds, uses a dark theme optimized for monitoring environments, spans the last hour of data by default, and includes flexible refresh intervals from 5 seconds to 1 day, making it an essential tool for operators to monitor system health, performance trends, and quickly identify issues in the MLOps pipeline.

Step 7: Deployment and Testing

7.1 Trigger the CI/CD Pipeline

Now let's deploy to AWS using GitHub Actions!

```
# Make sure all your changes are committed
git add .
git commit -m "Add all configuration files"
```

```
# Push to GitHub - this triggers the deployment!
git push
```

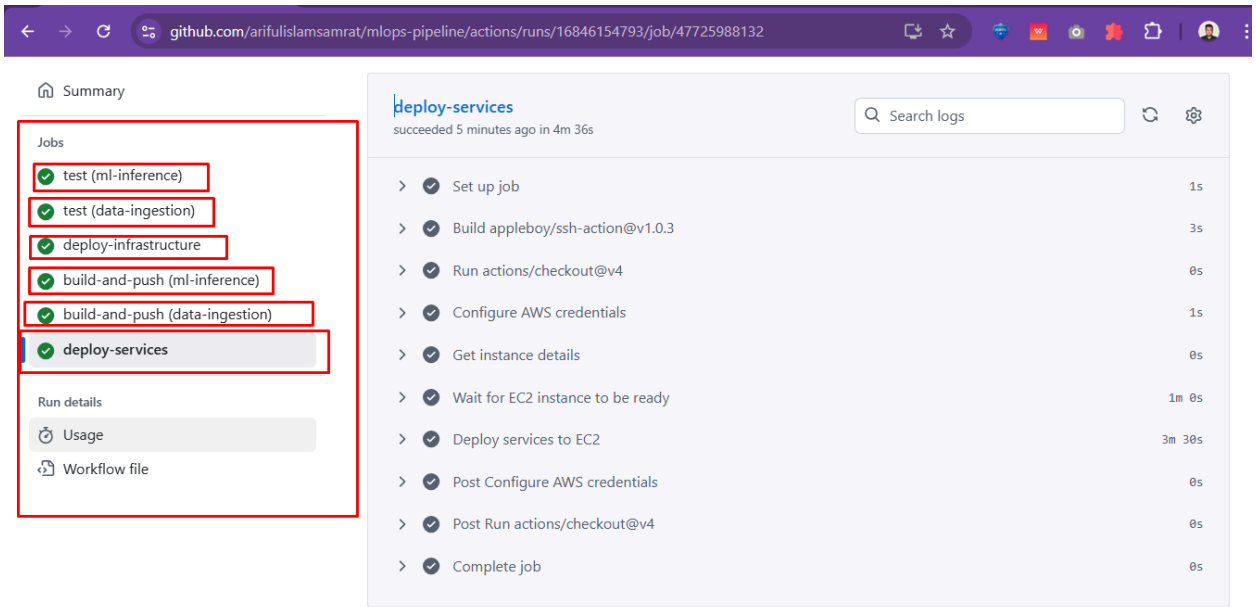
Monitor the deployment:

1. Go to your GitHub repository in browser
2. Click on "Actions" tab
3. You should see a workflow running
4. Click on it to see details
5. Watch each job complete (green checkmark = success)

The workflow will:

1. Run tests
2. Build Docker images
3. Push images to AWS ECR
4. Deploy infrastructure with Pulumi
5. Deploy services to EC2

This takes about 10-15 minutes total



After successfully deploying your MLOps pipeline, comprehensive testing is essential to validate all services are working correctly. This guide provides a complete Postman collection and step-by-step testing process to verify your ML Inference Service, Data Ingestion Service, and Monitoring Stack.

Step 7.2 : Download the Postman Collection

1.1 Create Collection File

Create a new file named `MLOps_Pipeline_Collection.json` and paste the following content:

```
"info":
      "_postman_id": "mlops-pipeline-comprehensive"
      "name": "MLOps Pipeline - Complete API Tests"
      "description": "Comprehensive testing suite for MLOps
Pipeline including ML Inference, Data Ingestion, Monitoring, and Load
Testing"
      "schema":
"https://schema.getpostman.com/json/collection/v2.1.0/collection.json
"
      "_exporter_id": "mlops-team"

"item":

            "name": "🤖 ML Inference Service"
            "item":

                  "name": "Health Check"
                  "event":

                        "listen": "test"
                        "script":
                              "exec":
                                    "pm.test(\"✅ ML
service is healthy\", function () {"
                                    "
pm.response.to.have.status(200);"
                                    "    const response =
pm.response.json();"
                                    "
pm.expect(response.status).to.eql(\"healthy\");"
                                    "
pm.expect(response.service).to.eql(\"ml-inference\");"
                                    "});"
                                    ""
```

"pm.test(\"⏱️ Response time is acceptable\", function () {"
"pm.expect(pm.response.responseTime).to.be.below(2000);"
"});"
""
"pm.test(\"📊 Content-Type is correct\", function () {"
"pm.expect(pm.response.headers.get(\"Content-Type\")).to.include(\"application/json\");"
"});"
""
"console.log(\"🎯 ML Inference Service Health Check: PASSED\");"

"type": "text/javascript"

"request":
"method": "GET"
"header":

"key": "Content-Type"
"value": "application/json"

"url":
"raw": "{{base_url}}:8001/health"
"host":
"{{base_url}}"

"port": "8001"
"path":
"health"

"description": "Validates that the ML Inference service is running and healthy"

"response":

```
                    "name": "Make Prediction"
                    "event":

                              "listen": "test"
                              "script":
                                   "exec":
                                        "pm.test(\"✅
Prediction request successful\", function () {"
                                        "
pm.response.to.have.status(200);"
                                        "});"
                                        ""
                                        "pm.test(\"📊
Response contains all required fields\", function () {"
                                        "    const response =
pm.response.json();"
                                        "
pm.expect(response).to.have.property('prediction');"
                                        "
pm.expect(response).to.have.property('model_version');"
                                        "
pm.expect(response).to.have.property('timestamp');"
                                        "});"
                                        ""
                                        "pm.test(\"🔢
Prediction is valid number\", function () {"
                                        "    const response =
pm.response.json();"
                                        "
pm.expect(response.prediction).to.be.a('number');"
                                        "
pm.expect(response.prediction).to.be.at.least(0);"
                                        "
pm.expect(response.prediction).to.be.at.most(1);"
                                        "});"
                                        ""
                                        "pm.test(\"🏷️ Model
version format is correct\", function () {"
                                        "    const response =
pm.response.json();"
                                        "
pm.expect(response.model_version).to.match(/^\\d+\\.\\d+\\.\\d+$/);"
                                        "});"
                                        ""
```

"pm.test(\"⏰ Timestamp is recent\", function () {"
"   const response = pm.response.json();"
"   const responseTime = new Date(response.timestamp * 1000);"
"   const currentTime = new Date();"
"   const timeDiff = currentTime - responseTime;"
"pm.expect(timeDiff).to.be.below(30000); // Within 30 seconds"
"});"
""
"// Store prediction for reporting"
"const response = pm.response.json();"
"pm.environment.set(\"last_prediction\", response.prediction);"
"console.log(`🎯 ML Prediction Result: ${response.prediction} (v${response.model_version})`);"

"type": "text/javascript"

"request":
"method": "POST"
"header":

"key": "Content-Type"
"value": "application/json"

"body":
"mode": "raw"
"raw": "{\n \"features\": [1.5, 2.3, 4.1, 0.8]\n}"

"url":
"raw": "{{base_url}}:8001/predict"

"host":
"{{base_url}}"

"port": "8001"
"path":
"predict"

"description": "Sends feature data to ML model and validates prediction response"

"response":

"name": "Get ML Metrics"
"event":

"listen": "test"
"script":
"exec":
"pm.test(\"✅ Metrics endpoint accessible\", function () {"
" pm.response.to.have.status(200);"
"});"
""
"pm.test(\"📈 Contains ML prediction metrics\", function () {"
" const body = pm.response.text();"
" pm.expect(body).to.include('ml_predictions_total');"
" pm.expect(body).to.include('ml_prediction_duration_seconds');"
"});"
""
"pm.test(\"📊 Content type is Prometheus format\", function () {"
" pm.expect(pm.response.headers.get('Content-Type')).to.include('text/plain');"
"});"
""
"pm.test(\"🔍 Metrics format is valid\", function () {"

```
                                                    "    const body =
pm.response.text();"
                                                    "    const lines =
body.split('\\n').filter(line => line.trim() !== '');"
                                                    "    let
hasValidMetrics = false;"
                                                    "
lines.forEach(line => {"
                                                    "        if
(!line.startsWith('#') && line.includes('ml_predictions_total')) {"
                                                    "
hasValidMetrics = true;"
                                                    "        }"
                                                    "    });"
                                                    "
pm.expect(hasValidMetrics).to.be.true;"
                                                    "});"
                                                    ""
                                                    "console.log(\"📊 ML
Metrics endpoint validated successfully\");"

                                          "type": "text/javascript"


                                "request":
                                     "method": "GET"
                                     "header":
                                     "url":
                                          "raw":
"{{base_url}}:8001/metrics"
                                          "host":
                                               "{{base_url}}"

                                          "port": "8001"
                                          "path":
                                               "metrics"


                                     "description": "Retrieves Prometheus
metrics from ML Inference service"

                                "response":
```

```
                "description": "Complete test suite for ML Inference
Service functionality"


                "name": "📥 Data Ingestion Service"
                "item":

                          "name": "Health Check"
                          "event":

                                    "listen": "test"
                                    "script":
                                          "exec":
                                                "pm.test(\"✅ Data
service is healthy\", function () {"
                                                "
pm.response.to.have.status(200);"
                                                "    const response =
pm.response.json();"
                                                "
pm.expect(response.status).to.eql(\"healthy\");"
                                                "
pm.expect(response.service).to.eql(\"data-ingestion\");"
                                                "});"
                                                ""
                                                "pm.test(\"⏱️
Response time is acceptable\", function () {"
                                                "
pm.expect(pm.response.responseTime).to.be.below(2000);"
                                                "});"
                                                ""
                                                "console.log(\"🎯
Data Ingestion Service Health Check: PASSED\");"

                                          "type": "text/javascript"



                          "request":
                               "method": "GET"
                               "header":

                                          "key": "Content-Type"
                                          "value":
"application/json"
```

```
                                "url":
                                     "raw":
"{{base_url}}:8002/health"
                                     "host":
                                          "{{base_url}}"

                                     "port": "8002"
                                     "path":
                                          "health"


                                "description": "Validates that the
Data Ingestion service is running and healthy"

                            "response":


                            "name": "Ingest Sample Data"
                            "event":

                                     "listen": "test"
                                     "script":
                                          "exec":
                                               "pm.test(\"✅ Data
ingestion successful\", function () {"
                                               "
pm.response.to.have.status(201);"
                                               "});"
                                               ""
                                               "pm.test(\"📊
Response has correct structure\", function () {"
                                               "    const response =
pm.response.json();"
                                               "
pm.expect(response).to.have.property('status');"
                                               "
pm.expect(response).to.have.property('id');"
                                               "
pm.expect(response).to.have.property('timestamp');"
                                               "
pm.expect(response.status).to.eql('success');"
                                               "});"
                                               ""
```

"pm.test(\"🔢 Generated ID is valid\", function () {"
"   const response = pm.response.json();"
"pm.expect(response.id).to.be.a('number');"
"pm.expect(response.id).to.be.above(0);"
"});"
""
"pm.test(\"⏰ Timestamp format is valid\", function () {"
"   const response = pm.response.json();"
"pm.expect(response.timestamp).to.be.a('string');"
"   pm.expect(new Date(response.timestamp)).to.be.a('date');"
"});"
""
"// Store ID for retrieval test"
"const response = pm.response.json();"
"pm.environment.set('last_data_id', response.id);"
"console.log(`📥 Data Ingested Successfully - ID: ${response.id}`);"

"type": "text/javascript"

"request":
"method": "POST"
"header":

"key": "Content-Type"
"value": "application/json"

"body":
"mode": "raw"

"raw": "{\n \"sensor\": \"temperature_sensor_01\",\n \"value\": 25.5,\n \"location\": \"warehouse_a\",\n \"metadata\": {\n   \"unit\": \"celsius\",\n \"calibration_date\": \"2025-01-01\"\n },\n \"timestamp\": \"{{$isoTimestamp}}\"\n}"

"url":
"raw": "{{base_url}}:8002/ingest"
"host":
"{{base_url}}"

"port": "8002"
"path":
"ingest"

"description": "Ingests sample sensor data and validates the response"

"response":

"name": "Retrieve Data by ID"
"event":

"listen": "prerequest"
"script":
"exec":
"// Use stored ID or default to 1"
"const dataId = pm.environment.get('last_data_id') || '1';"
"pm.environment.set('current_data_id', dataId);"
"console.log(`🔍 Retrieving data with ID: ${dataId}`);"
"type": "text/javascript"

"listen": "test"
"script":
"exec":

```
                                                    "pm.test(\"✅ Data
retrieval successful\", function () {"
                                                    "
pm.response.to.have.status(200);"
                                                    "});"
                                                    ""
                                                    "pm.test(\"📊
Retrieved data has correct structure\", function () {"
                                                    "    const response =
pm.response.json();"
                                                    "
pm.expect(response).to.have.property('data');"
                                                    "
pm.expect(response).to.have.property('timestamp');"
                                                    "
pm.expect(response).to.have.property('source_ip');"
                                                    "
pm.expect(response).to.have.property('id');"
                                                    "});"
                                                    ""
                                                    "pm.test(\"🎯
Retrieved ID matches requested\", function () {"
                                                    "    const response =
pm.response.json();"
                                                    "    const
requestedId = parseInt(pm.environment.get('current_data_id'));"
                                                    "
pm.expect(response.id).to.eql(requestedId);"
                                                    "});"
                                                    ""
                                                    "pm.test(\"📋 Data
content is preserved\", function () {"
                                                    "    const response =
pm.response.json();"
                                                    "
pm.expect(response.data).to.be.an('object');"
                                                    "
pm.expect(response.data).to.have.property('sensor');"
                                                    "});"
                                                    ""
                                                    "console.log(\"📤
Data Retrieved Successfully\");"

                                                "type": "text/javascript"
```

```
                    "request":
                        "method": "GET"
                        "header":
                        "url":
                            "raw":
"{{base_url}}:8002/data/{{current_data_id}}"
                            "host":
                                "{{base_url}}"

                            "port": "8002"
                            "path":
                                "data"
                                "{{current_data_id}}"


                        "description": "Retrieves a specific
data record by ID and validates the content"

                    "response":


                    "name": "List Recent Data"
                    "event":

                            "listen": "test"
                            "script":
                                "exec":
                                    "pm.test(\"✅ Data
listing successful\", function () {"
                                    "
pm.response.to.have.status(200);"
                                    "});"
                                    ""
                                    "pm.test(\"📊
Response has correct structure\", function () {"
                                    "    const response =
pm.response.json();"
                                    "
pm.expect(response).to.have.property('total');"
                                    "
pm.expect(response).to.have.property('data');"
                                    "
pm.expect(response.data).to.be.an('array');"
```

```
                                                    "});"
                                                    ""
                                                    "pm.test(\"🔢
Respects limit parameter\", function () {"
                                                    "    const response =
pm.response.json();"
                                                    "
pm.expect(response.data.length).to.be.at.most(5);"
                                                    "});"
                                                    ""
                                                    "pm.test(\"📋 Each
data item has required fields\", function () {"
                                                    "    const response =
pm.response.json();"
                                                    "    if
(response.data.length > 0) {"
                                                    "
response.data.forEach(item => {"
                                                    "
pm.expect(item).to.have.property('id');"
                                                    "
pm.expect(item).to.have.property('timestamp');"
                                                    "
pm.expect(item).to.have.property('data');"
                                                    "        });"
                                                    "    }"
                                                    "});"
                                                    ""
                                                    "const response =
pm.response.json();"
                                                    "console.log(`📋
Listed ${response.data.length} data records (Total:
${response.total})`);"

                                                "type": "text/javascript"



                                "request":
                                     "method": "GET"
                                     "header":
                                     "url":
                                          "raw":
"{{base_url}}:8002/data?limit=5"
                                          "host":
```

"{{base_url}}"

"port": "8002"
"path":
"data"

"query":

"key": "limit"
"value": "5"
"description": "Limit number of records returned"

"description": "Retrieves a list of recent data records with pagination"

"response":

"name": "Get Data Metrics"
"event":

"listen": "test"
"script":
"exec":
"pm.test(\"✅ Data metrics endpoint accessible\", function () {"
"pm.response.to.have.status(200);"
"});"
""
"pm.test(\"📈 Contains data ingestion metrics\", function () {"
"   const body = pm.response.text();"
"pm.expect(body).to.include('data_ingestion_total');"
"pm.expect(body).to.include('data_ingestion_duration_seconds');"
"pm.expect(body).to.include('data_size_bytes');"
"});"
""

```
                                                "pm.test(\"📊 Metrics
format is valid\", function () {"
                                                "    const body =
pm.response.text();"
                                                "    const lines =
body.split('\\n').filter(line => line.trim() !== '');"
                                                "    let
hasValidMetrics = false;"
                                                "
lines.forEach(line => {"
                                                "        if
(!line.startsWith('#') && line.includes('data_ingestion_total')) {"
                                                "
hasValidMetrics = true;"
                                                "        }"
                                                "    });"
                                                "
pm.expect(hasValidMetrics).to.be.true;"
                                                "});"
                                                ""
                                                "console.log(\"📊
Data Ingestion metrics validated successfully\");"

                                            "type": "text/javascript"



                                "request":
                                     "method": "GET"
                                     "header":
                                     "url":
                                          "raw":
"{{base_url}}:8002/metrics"
                                          "host":
                                               "{{base_url}}"

                                          "port": "8002"
                                          "path":
                                               "metrics"


                                     "description": "Retrieves Prometheus
metrics from Data Ingestion service"

                                "response":
```

```
                "description": "Complete test suite for Data
Ingestion Service functionality"


                "name": "📈 Monitoring Services"
                "item":

                          "name": "Prometheus Health Check"
                          "event":

                                    "listen": "test"
                                    "script":
                                         "exec":
                                              "pm.test(\"✅
Prometheus is accessible\", function () {"
                                              "
pm.response.to.have.status(200);"
                                              "});"
                                              ""
                                              "pm.test(\"📊
Prometheus returns valid response\", function () {"
                                              "    const response =
pm.response.json();"
                                              "
pm.expect(response).to.have.property('status');"
                                              "
pm.expect(response.status).to.eql('success');"
                                              "
pm.expect(response).to.have.property('data');"
                                              "});"
                                              ""
                                              "pm.test(\"🎯
Services are being monitored\", function () {"
                                              "    const response =
pm.response.json();"
                                              "    const results =
response.data.result;"
                                              "
pm.expect(results).to.be.an('array');"
                                              "
pm.expect(results.length).to.be.above(0);"
                                              "});"
                                              ""
```

"// Count active services"
"const response = pm.response.json();"
"const activeServices = response.data.result.filter(result => result.value[1] === '1');"
"console.log(`📈 Prometheus monitoring ${activeServices.length} active services`);"

"type": "text/javascript"

"request":
"method": "GET"
"header":
"url":
"raw": "{{base_url}}:9090/api/v1/query?query=up"
"host":
"{{base_url}}"

"port": "9090"
"path":
"api"
"v1"
"query"

"query":

"key": "query"
"value": "up"
"description": "Query all service health status"

"description": "Validates Prometheus is running and monitoring all services"

"response":

"name": "Grafana Health Check"
"event":

```
                                        "listen": "test"
                                        "script":
                                             "exec":
                                                  "pm.test(\"✅ Grafana
is accessible\", function () {"
                                                  "
pm.response.to.have.status(200);"
                                                  "});"
                                                  ""
                                                  "pm.test(\"💾
Database connection is healthy\", function () {"
                                                  "    const response =
pm.response.json();"
                                                  "
pm.expect(response).to.have.property('database');"
                                                  "
pm.expect(response.database).to.eql('ok');"
                                                  "});"
                                                  ""
                                                  "pm.test(\"🔧 Grafana
version is present\", function () {"
                                                  "    const response =
pm.response.json();"
                                                  "
pm.expect(response).to.have.property('version');"
                                                  "
pm.expect(response.version).to.be.a('string');"
                                                  "});"
                                                  ""
                                                  "const response =
pm.response.json();"
                                                  "console.log(`📊
Grafana v${response.version} is healthy and accessible`);"

                                             "type": "text/javascript"



                                  "request":
                                       "method": "GET"
                                       "header":
                                       "url":
                                            "raw":
"{{base_url}}:3000/api/health"
```

"host":
"{{base_url}}"

"port": "3000"
"path":
"api"
"health"

"description": "Validates Grafana dashboard service health and connectivity"

"response":

"name": "Check ML Service Metrics"
"event":

"listen": "test"
"script":
"exec":
"pm.test(\"✅ ML metrics query successful\", function () {"
"pm.response.to.have.status(200);"
"});"
""
"pm.test(\"📊 ML prediction metrics available\", function () {"
"   const response = pm.response.json();"
"pm.expect(response.status).to.eql('success');"
"pm.expect(response.data.result).to.be.an('array');"
"});"
""
"// Extract prediction count"
"const response = pm.response.json();"
"if (response.data.result.length > 0) {"
"   const predictionCount = response.data.result[0].value[1];"

"   console.log(`🤖 Total ML Predictions: ${predictionCount}`);"
"} else {"
"   console.log(\"🤖 No ML predictions recorded yet\");"
"}"

"type": "text/javascript"

"request":
"method": "GET"
"header":
"url":
"raw": "{{base_url}}:9090/api/v1/query?query=ml_predictions_total"
"host":
"{{base_url}}"

"port": "9090"
"path":
"api"
"v1"
"query"

"query":

"key": "query"
"value": "ml_predictions_total"
"description": "Query total ML predictions made"

"description": "Queries Prometheus for ML service metrics to validate monitoring integration"

"response":

"description": "Validation tests for monitoring and observability infrastructure"

```
                "name": "⚡ Performance & Load Testing"
                "item":

                          "name": "ML Inference Performance Test"
                          "event":

                                    "listen": "test"
                                    "script":
                                          "exec":
                                                "pm.test(\"✅
Performance test - prediction successful\", function () {"
                                                "
pm.response.to.have.status(200);"
                                                "});"
                                                ""
                                                "pm.test(\"⚡
Performance test - response time acceptable\", function () {"
                                                "
pm.expect(pm.response.responseTime).to.be.below(5000);"
                                                "});"
                                                ""
                                                "pm.test(\"🔢
Performance test - valid prediction returned\", function () {"
                                                "    const response =
pm.response.json();"
                                                "
pm.expect(response).to.have.property('prediction');"
                                                "
pm.expect(response.prediction).to.be.a('number');"
                                                "});"
                                                ""
                                                "// Track response
times for performance analysis"
                                                "pm.test(\"📊 Record
performance metrics\", function () {"
                                                "    const
responseTime = pm.response.responseTime;"
                                                "    const
currentTimes = pm.environment.get('ml_response_times') || '[]';"
                                                "    const times =
JSON.parse(currentTimes);"
                                                "
times.push(responseTime);"
                                                "    "
```

```
                                                    "    // Keep only
last 10 response times"
                                                    "    if (times.length
> 10) {"
                                                    "
times.shift();"
                                                    "    }"
                                                    "    "
                                                    "
pm.environment.set('ml_response_times', JSON.stringify(times));"
                                                    "    "
                                                    "    // Calculate
statistics"
                                                    "    const avg =
times.reduce((a, b) => a + b, 0) / times.length;"
                                                    "    const max =
Math.max(...times);"
                                                    "    const min =
Math.min(...times);"
                                                    "    "
                                                    "    console.log(`⚡
ML Performance Stats: Avg: ${avg.toFixed(2)}ms, Min: ${min}ms, Max:
${max}ms`);"
                                                    "});"

                                            "type": "text/javascript"



                              "request":
                                   "method": "POST"
                                   "header":

                                            "key": "Content-Type"
                                            "value":
"application/json"


                                   "body":
                                        "mode": "raw"
                                        "raw": "{\n  \"features\": [\n
{{$randomInt}},\n    {{$randomInt}},\n    {{$randomInt}},\n
{{$randomInt}}\n  ]\n}"

                                   "url":
```

```
                                        "raw":
"{{base_url}}:8001/predict"
                                        "host":
                                             "{{base_url}}"

                                        "port": "8001"
                                        "path":
                                             "predict"


                                   "description": "Performance test with
random data to measure ML inference response times"

                              "response":


                              "name": "Data Ingestion Performance Test"
                              "event":

                                        "listen": "test"
                                        "script":
                                             "exec":
                                                  "pm.test(\"✅
Performance test - ingestion successful\", function () {"
                                                  "
pm.response.to.have.status(201);"
                                                  "});"
                                                  ""
                                                  "pm.test(\"⚡
Performance test - response structure valid\", function () {"
                                                  "    const response =
pm.response.json();"
                                                  "
pm.expect(response).to.have.property('status');"
                                                  "
pm.expect(response.status).to.eql('success');"
                                                  "});"
                                                  ""
                                                  "// Track ingestion
rate and performance"
                                                  "pm.test(\"📊 Track
ingestion performance\", function () {"
                                                  "    const
responseTime = pm.response.responseTime;"
```

```
"    const currentTime = Date.now();"
"    const lastTime = pm.environment.get('last_ingestion_time') || currentTime;"
"    const timeDiff = currentTime - lastTime;"
"    "
"pm.environment.set('last_ingestion_time', currentTime);"
"    "
"    // Calculate ingestion rate"
"    if (timeDiff > 0) {"
"        const rate = 1000 / timeDiff; // requests per second"
"console.log(`📥 Ingestion Rate: ${rate.toFixed(2)} req/sec, Response Time: ${responseTime}ms`);"
"    }"
"    "
"    // Track response times"
"    const currentTimes = pm.environment.get('data_response_times') || '[]';"
"    const times = JSON.parse(currentTimes);"
"times.push(responseTime);"
"    "
"    if (times.length > 10) {"
"times.shift();"
"    }"
"    "
"pm.environment.set('data_response_times', JSON.stringify(times));"
"});"

"type": "text/javascript"


"request":
```

```
                                "method": "POST"
                                "header":

                                          "key": "Content-Type"
                                          "value":
"application/json"


                                "body":
                                     "mode": "raw"
                                     "raw": "{\n  \"sensor\":
\"load_test_{{$randomInt}}\",\n  \"value\": {{$randomInt}},\n
\"batch_id\": \"{{$guid}}\",\n  \"location\":
\"test_facility_{{$randomInt}}\",\n  \"timestamp\":
\"{{$isoTimestamp}}\"\n}"

                                "url":
                                     "raw":
"{{base_url}}:8002/ingest"
                                     "host":
                                          "{{base_url}}"

                                     "port": "8002"
                                     "path":
                                          "ingest"


                                "description": "Performance test for
data ingestion with random payloads"

                           "response":


                           "name": "Concurrent Load Test"
                           "event":

                                     "listen": "test"
                                     "script":
                                          "exec":
                                               "pm.test(\"✅
Concurrent load test successful\", function () {"
                                               "
pm.response.to.have.status(200);"
                                               "});"
                                               ""
```

"pm.test(\"⚡ Load test - system remains responsive\", function () {"
"pm.expect(pm.response.responseTime).to.be.below(10000);"
"});"
""
"// Performance tracking for load testing"
"const responseTime = pm.response.responseTime;"
"const testNumber = pm.environment.get('load_test_count') || 0;"
"pm.environment.set('load_test_count', parseInt(testNumber) + 1);"
""
"console.log(`🔥 Load Test #${parseInt(testNumber) + 1}: ${responseTime}ms`);"

"type": "text/javascript"

"request":
"method": "POST"
"header":

"key": "Content-Type"
"value": "application/json"

"body":
"mode": "raw"
"raw": "{\n \"features\": [\n   {{$randomFloat}},\n   {{$randomFloat}},\n   {{$randomFloat}},\n   {{$randomFloat}}\n]\n}"

"url":
"raw": "{{base_url}}:8001/predict"
"host":
"{{base_url}}"

"port": "8001"
"path":

"predict"

"description": "Concurrent load testing to validate system performance under stress"

"response":

"description": "Performance and load testing scenarios to validate system scalability"

"name": "🚨 Error Handling Tests"
"item":

"name": "Invalid ML Prediction Request"
"event":

"listen": "test"
"script":
"exec":
"pm.test(\"✅ Invalid request properly rejected\", function () {"
"pm.response.to.have.status(400);"
"});"
""
"pm.test(\"📝 Error response has correct structure\", function () {"
"   const response = pm.response.json();"
"pm.expect(response).to.have.property('error');"
"pm.expect(response.error).to.be.a('string');"
"pm.expect(response.error).to.include('features');"
"});"
""
"console.log(\"🚨 Invalid ML request properly handled\");"

"type": "text/javascript"

```
                    "request":
                        "method": "POST"
                        "header":

                            "key": "Content-Type"
                            "value":
"application/json"


                        "body":
                            "mode": "raw"
                            "raw": "{\n  \"invalid_field\":
\"test\"\n}"

                        "url":
                            "raw":
"{{base_url}}:8001/predict"
                            "host":
                                "{{base_url}}"

                            "port": "8001"
                            "path":
                                "predict"


                        "description": "Tests error handling
for invalid ML prediction requests"

                    "response":


                    "name": "Empty Data Ingestion"
                    "event":

                            "listen": "test"
                            "script":
                                "exec":
                                    "pm.test(\"✅ Empty
request properly rejected\", function () {"
                                    "
pm.response.to.have.status(400);"
                                    "});"
                                    ""
```

```
                                                    "pm.test(\"📝 Error
message is descriptive\", function () {"
                                                    "    const response =
pm.response.json();"
                                                    "
pm.expect(response).to.have.property('error');"
                                                    "
pm.expect(response.error).to.include('No data provided');"
                                                    "});"
                                                    ""
                                                    "console.log(\"🚨
Empty data request properly handled\");"

                                          "type": "text/javascript"


                              "request":
                                   "method": "POST"
                                   "header":

                                          "key": "Content-Type"
                                          "value":
"application/json"


                                   "body":
                                        "mode": "raw"
                                        "raw": "{}"

                                   "url":
                                        "raw":
"{{base_url}}:8002/ingest"
                                        "host":
                                             "{{base_url}}"

                                        "port": "8002"
                                        "path":
                                             "ingest"


                                   "description": "Tests error handling
for empty data ingestion requests"

                              "response":
```

```
                    "name": "Non-existent Data Retrieval"
                    "event":

                              "listen": "test"
                              "script":
                                   "exec":
                                        "pm.test(\"✅
Non-existent data returns 404\", function () {"
                                        "
pm.response.to.have.status(404);"
                                        "});"
                                        ""
                                        "pm.test(\"📝 404
response has error message\", function () {"
                                        "    const response =
pm.response.json();"
                                        "
pm.expect(response).to.have.property('error');"
                                        "
pm.expect(response.error).to.include('not found');"
                                        "});"
                                        ""
                                        "console.log(\"🚨
Non-existent data request properly handled\");"

                                   "type": "text/javascript"



                    "request":
                         "method": "GET"
                         "header":
                         "url":
                              "raw":
"{{base_url}}:8002/data/99999"
                              "host":
                                   "{{base_url}}"

                              "port": "8002"
                              "path":
                                   "data"
                                   "99999"
```

"description": "Tests error handling for requests to non-existent data records"

"response":

"description": "Error handling and edge case validation tests"

"event":

"listen": "prerequest"
"script":
"type": "text/javascript"
"exec":
"// Collection-level pre-request script"
"pm.globals.set('test_start_time', new Date().toISOString());"
""
"// Generate random test data"
"pm.globals.set('random_sensor_value', Math.random() * 100);"
"pm.globals.set('random_features', JSON.stringify(["
"   Math.random() * 10,"
"   Math.random() * 10,"
"   Math.random() * 10,"
"   Math.random() * 10"
"]));"
""
"// Initialize test counters"
"if (!pm.environment.get('tests_run')) {"
"   pm.environment.set('tests_run', 0);"
"   pm.environment.set('tests_passed', 0);"
"   pm.environment.set('tests_failed', 0);"
"}"

"listen": "test"

```
                "script":
                     "type": "text/javascript"
                     "exec":
                          "// Collection-level test tracking"
                          "const testCount =
parseInt(pm.environment.get('tests_run') || 0) + 1;"
                          "pm.environment.set('tests_run',
testCount);"
                          ""
                          "// Track pass/fail status"
                          "const results = pm.response.json ?
pm.response.code : 0;"
                          "if (pm.response.code >= 200 &&
pm.response.code < 400) {"
                          "    const passed =
parseInt(pm.environment.get('tests_passed') || 0) + 1;"
                          "    pm.environment.set('tests_passed',
passed);"
                          "} else {"
                          "    const failed =
parseInt(pm.environment.get('tests_failed') || 0) + 1;"
                          "    pm.environment.set('tests_failed',
failed);"
                          "}"



     "variable":

                "key": "base_url"
                "value": "http://3.0.199.135"
                "type": "string"
                "description": "Base URL for MLOps Pipeline services"


                "key": "api_version"
                "value": "v1"
                "type": "string"
                "description": "API version for service endpoints"


}
```

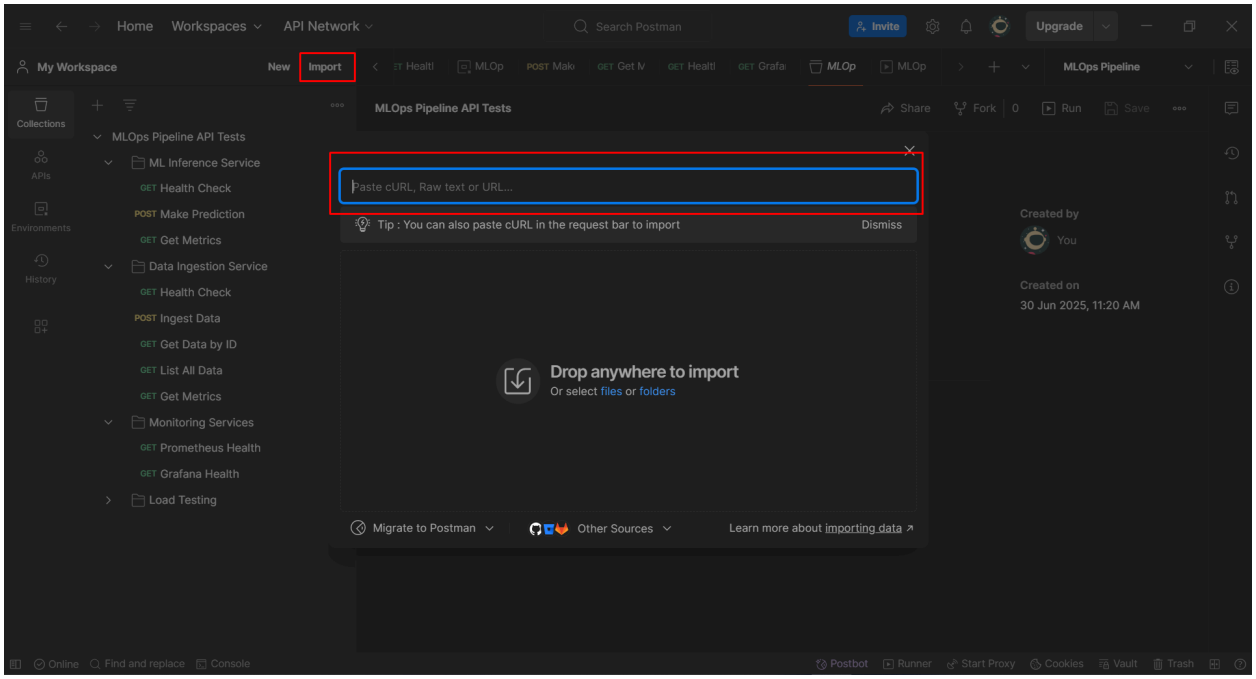
Step 7.3 : Import Collection into Postman

7.3.1 Save the Collection File

1. Create a new file on your computer named `MLOps_Pipeline_Collection.json`
2. Copy the entire JSON content from Step 1.1 above
3. Paste it into the file and save it

7.3.2 Import into Postman

1. Open Postman (Desktop or Web application)
2. Click "Import" button in the top-left corner
3. Select "Upload Files" or drag and drop the JSON file
4. Choose `MLOps_Pipeline_Collection.json`
5. Click "Import" - you should see the collection appear in your sidebar



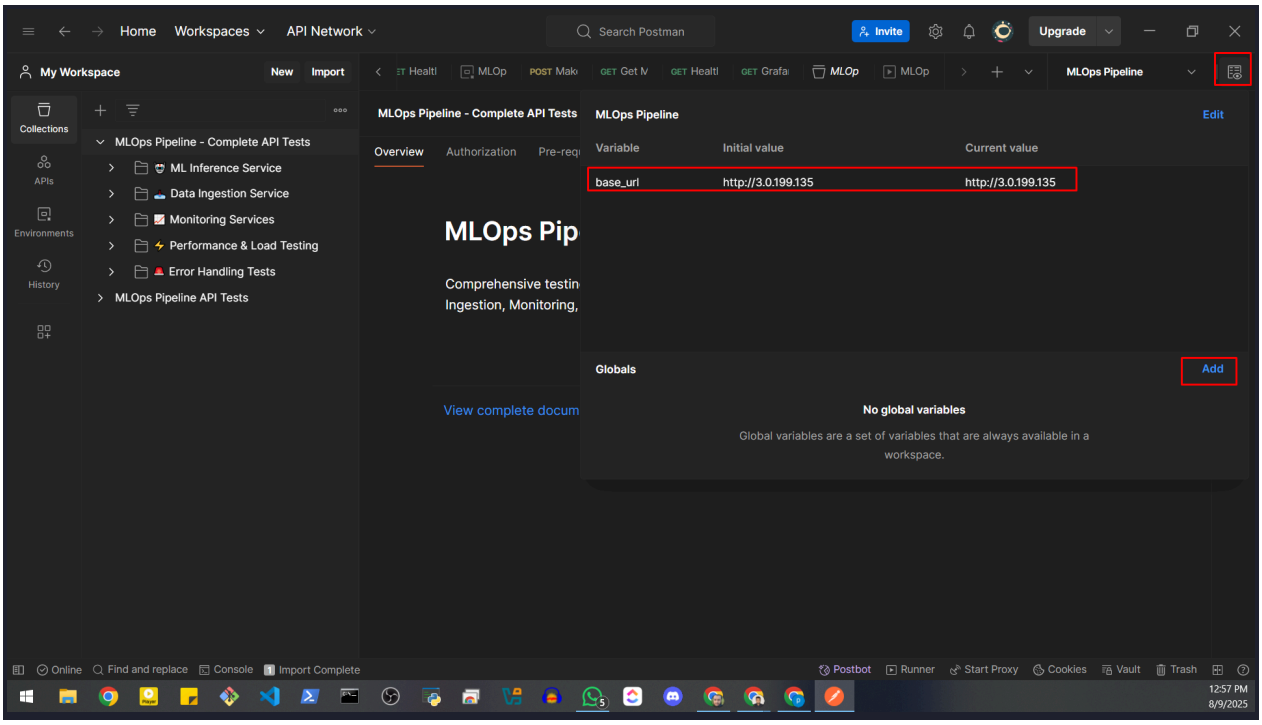
7.3.3 Verify Import Success

- ✅ Collection name: "MLOps Pipeline - Complete API Tests"
- ✅ 5 main folders with emojis
- ✅ Total of 15+ individual test requests
- ✅ Environment variable `base_url` set to your IP

7.3.4 : Configure Environment

3.1 Update Base URL (Critical Step)

1. Click on the environment dropdown (top-right corner)
2. Select "MLOps Pipeline - Complete API Tests" environment
3. Click the eye icon 👁️ to view variables
4. Click "Edit"
5. Update `base_url` to your actual EC2 IP address: Replace with your actual IP if different
   `Current Value: http://3.0.199.135`



3.2 Verify Environment Variables

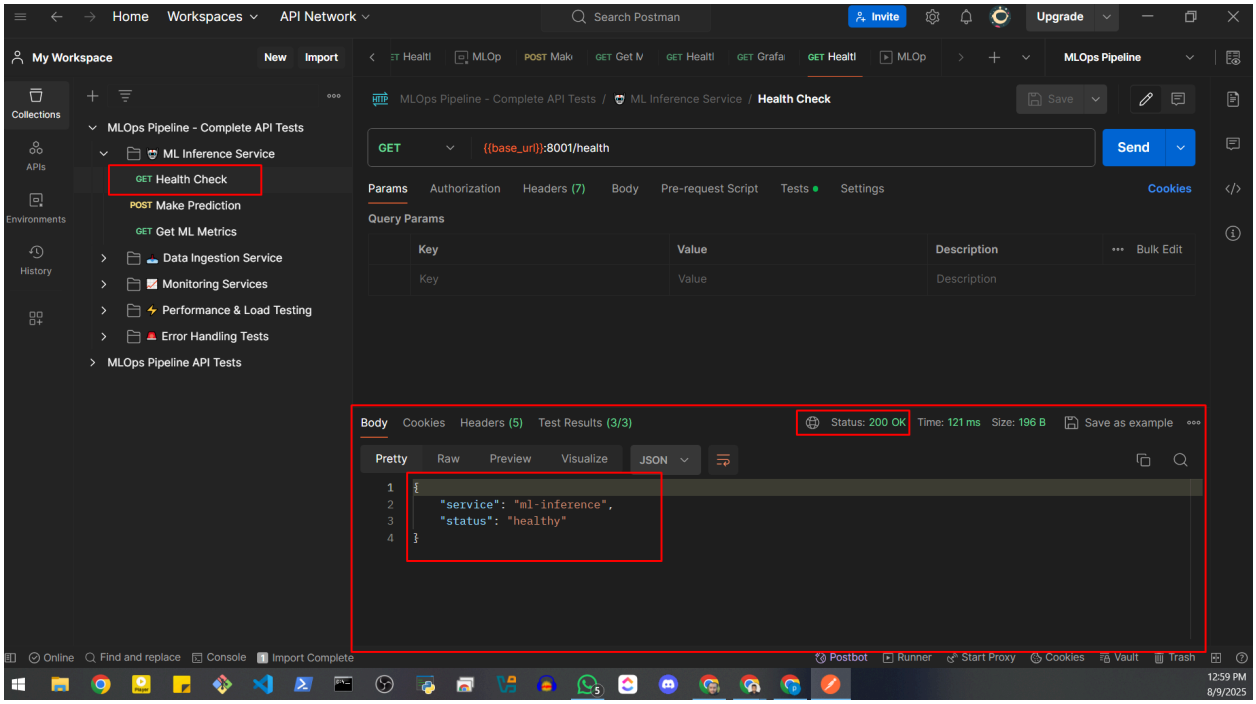
- ✅ `base_url`: Your EC2 public IP (e.g., `http://3.0.199.135`)
- ✅ `api_version`: v1
- ✅ Auto-generated variables will be created during testing

Step 7.4: Execute Complete Test Suite

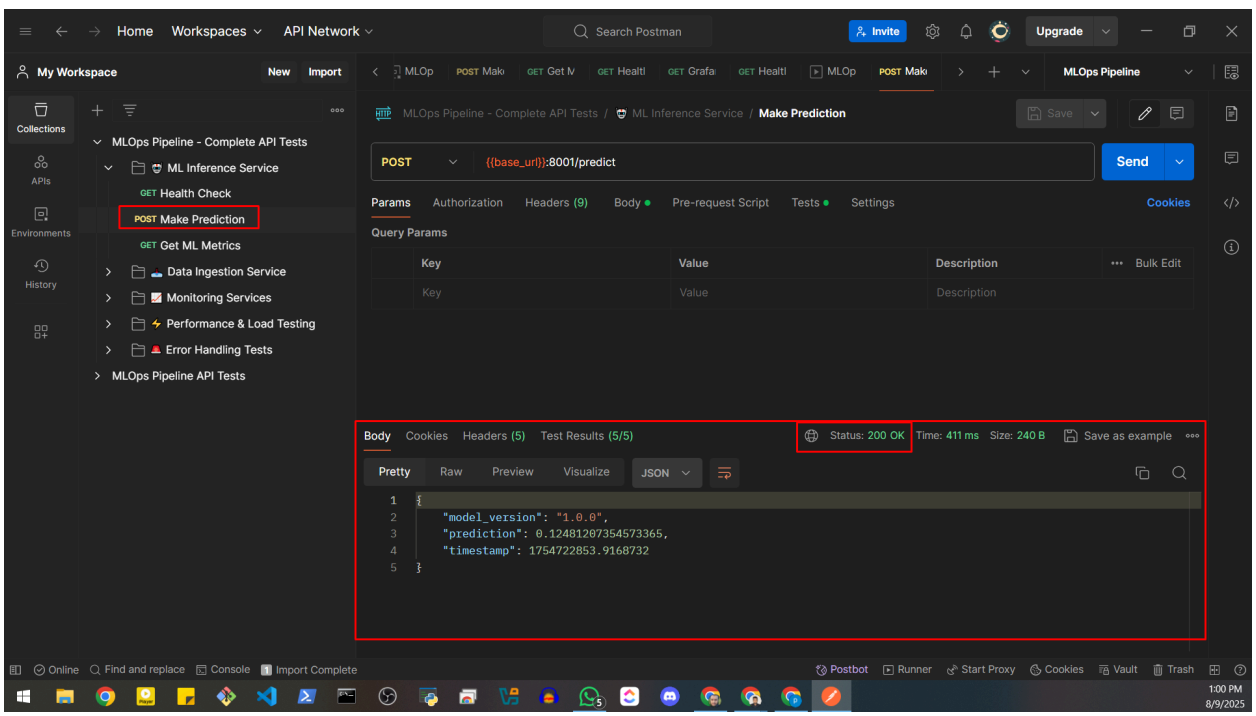
4.1 Run Individual Service Tests

Test ML Inference Service

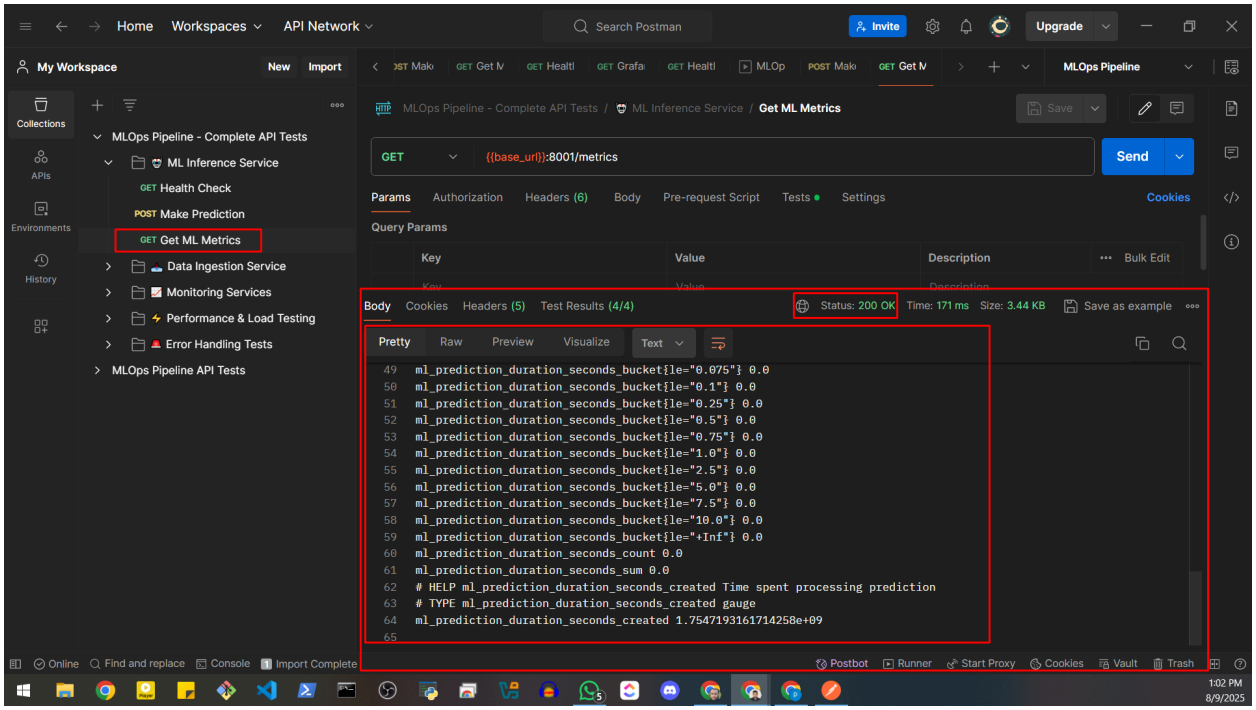
Expand "🤖 ML Inference Service" folder and Click "Health Check" → Send → Verify ✅ status



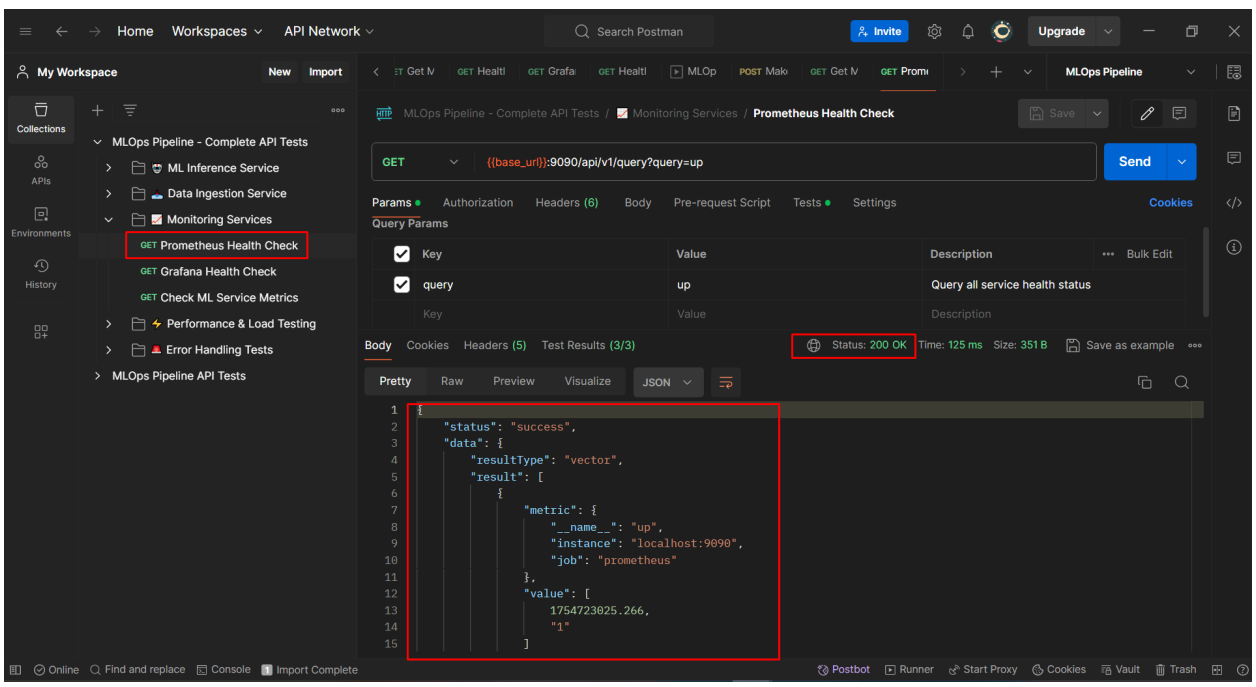
Now Click "Make Prediction" → Send → Check prediction value

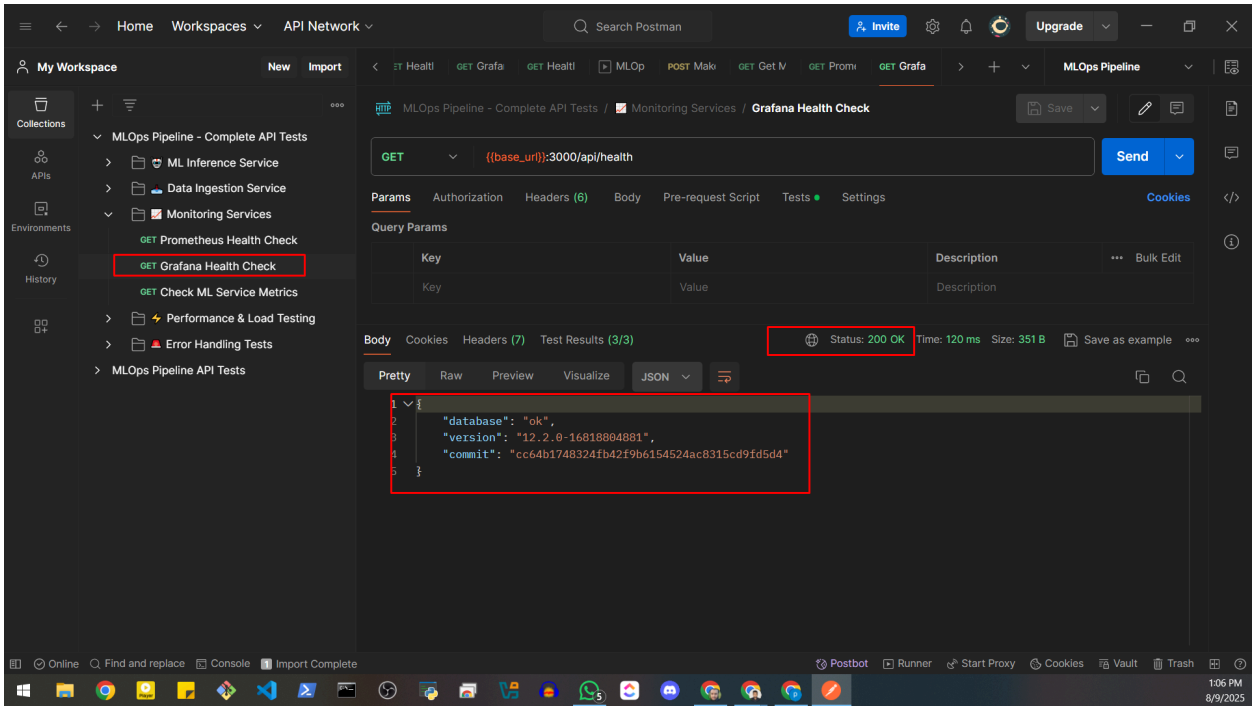


And the last one Click "Get ML Metrics" → Send → Verify metrics format

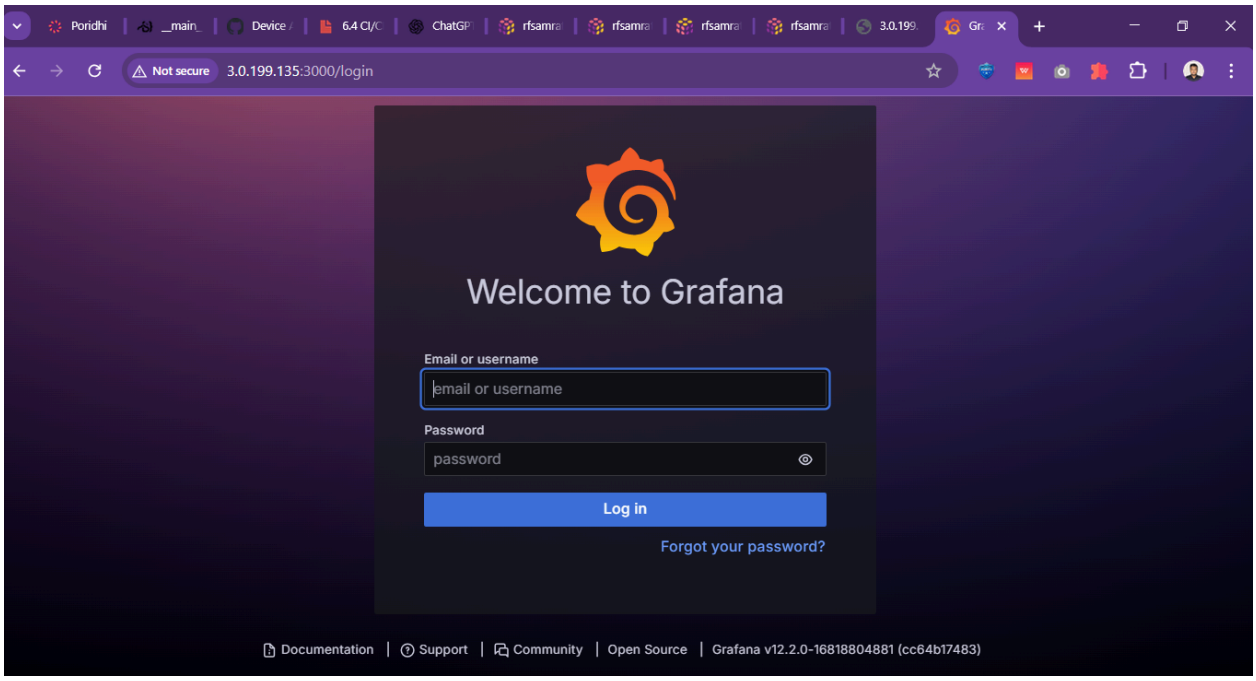


Monitoring services should also showed status 200 both in prometheus and grafana

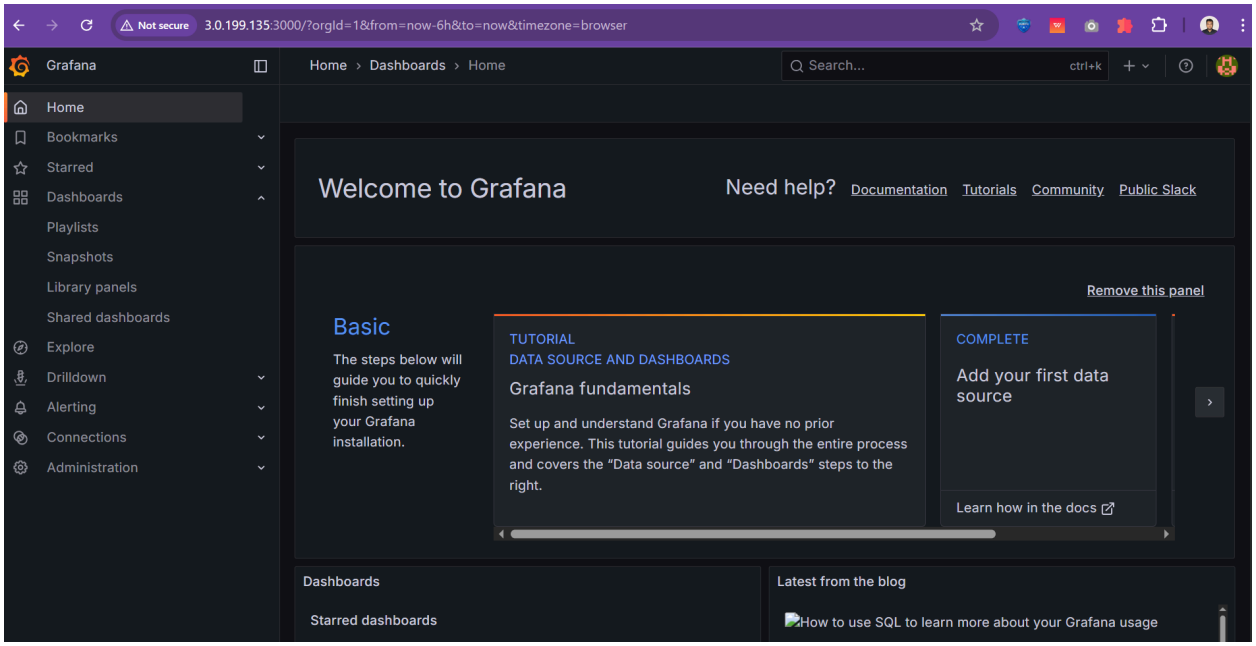




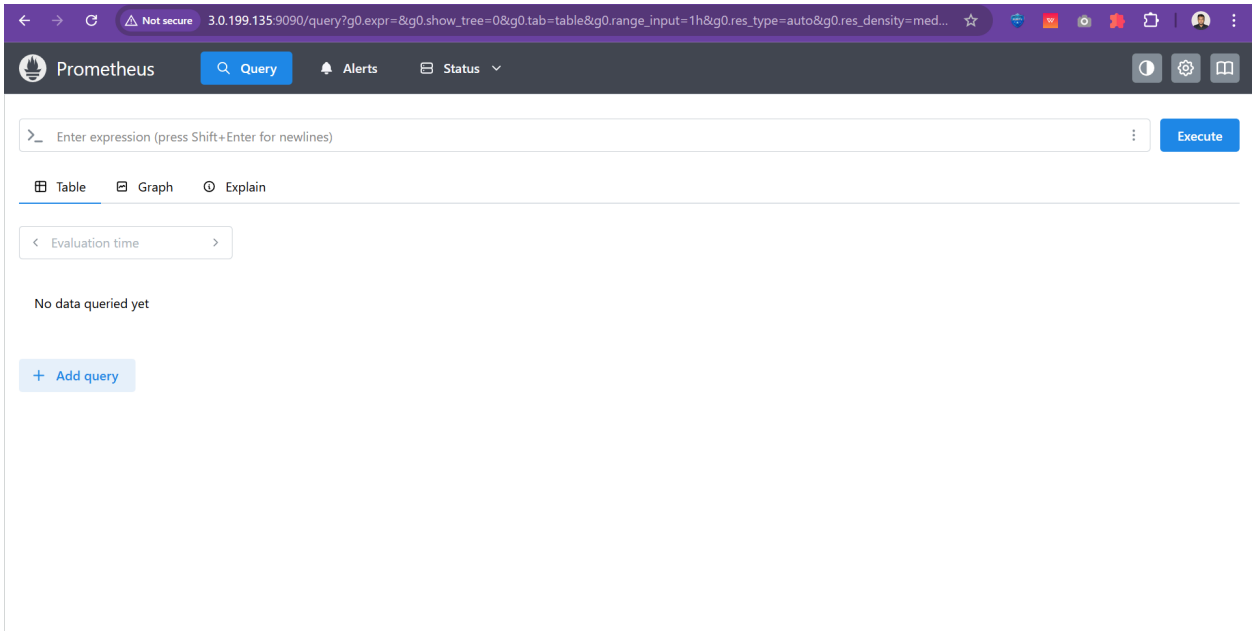
And if we go to the website we can also see the grafana dashboard and prometheus



initial password and username is admin , admin after logging in we will get access into grafana dashboard and can customize as needed .



also like grafana prometheus can be accessed in the 9090 endpoint .



Conclusion :

This comprehensive MLOps pipeline demonstrates a modern cloud-native approach to machine learning operations using Infrastructure as Code (IaC) and containerized microservices. The project implements a complete CI/CD workflow that automatically builds, tests, and deploys ML services to AWS with real-time monitoring capabilities.

: The pipeline follows a "destroy and recreate" deployment strategy using Pulumi for infrastructure management, ensuring clean, consistent environments on every deployment. Two core microservices handle ML inference (port 8001) and data ingestion (port 8002), both containerized with Docker and deployed to AWS EC2 instances. The system includes comprehensive monitoring through Prometheus metrics collection and Grafana dashboards for real-time observability.

Built with Python/Flask for services, Docker for containerization, Pulumi for AWS infrastructure provisioning, GitHub Actions for CI/CD automation, and Prometheus/Grafana for monitoring. The entire stack runs on AWS free tier resources (t2.micro instances) making it cost-effective for learning and development.

Automated testing at multiple levels (unit, integration, performance), parallel CI/CD pipeline execution, complete infrastructure automation, built-in monitoring and alerting, comprehensive API testing with Postman collections, and load testing capabilities. The pipeline emphasizes reliability and simplicity over complex deployment strategies, making it ideal for ML operations where consistency and observability are paramount.

: Complete working MLOps pipeline with automated deployment, comprehensive Postman testing suite with 15+ test scenarios, monitoring dashboards, performance benchmarking, error handling validation, and detailed documentation for setup, testing, and maintenance. The project serves as a production-ready template for modern MLOps implementations.